



Universitat
de les Illes Balears

TREBALL DE FI DE GRAU

APRENENTATGE PER REFORÇ PROFUND PER AL JOC DEL TRUC

Josep Ferriol Font

Grau d'Enginyeria Informàtica

Escola Politècnica Superior

Any acadèmic 2025-26

APRENTATGE PER REFORÇ PROFUND PER AL JOC DEL TRUC

Josep Ferriol Font

Treball de Fi de Grau

Escola Politècnica Superior

Universitat de les Illes Balears

Any acadèmic 2025-26

Paraules clau del treball: aprenentatge per reforç, truc, jocs de cartes, informació parcial, curriculum learning, self-play

Tutor: Dr. Francesc Xavier Gayà Morey

Cotutor: Dr. Gabriel Moyà Alcover

Segons estudis demogràfics, una gran part de la població mundial neix en contextos on l'accés a l'educació i la sanitat no estan garantits. Tenir la sort de néixer a Europa, en un estat de dret que reconeix la salut i l'educació com a pilars fonamentals de la societat, és un privilegi que no hauria de donar-se per descomptat. Per això, agrair haver tengut

l'oportunitat de cursar una educació pública i de qualitat a la meua terra, Mallorca.

A més, un agraïment especial dirigit a la meua família, els altres grans patrocinadors d'aquesta carrera. El seu suport incondicional ha estat el fonament sobre el qual s'ha construït tot el que he aconseguit.

SUMARI

Sumari	iii
Índex de figures	v
Índex de taules	vii
Llista d'acrònims	ix
Resum	xi
1 Introducció	1
1.1 La intel·ligència artificial en el joc	2
1.2 El repte de la informació parcial	2
1.3 El Truc: engany, parella i farol	3
1.4 Objectius i estructura d'aquest treball	3
2 Fonaments Teòrics i Treball Relacionat	5
2.1 Aprenentatge per Reforç	5
2.2 Estat de l'art en jocs d'informació parcial	9
2.3 Mètodes usats	10
2.3.1 Deep Q-Network	10
2.3.2 Proximal Policy Optimization	11
2.3.3 Deep Recurrent Q-Learning	12
2.3.4 Neural Fictitious Self-Play	12
3 El Joc del Truc	15
3.1 Regles del joc	15
3.2 Arquitectura del simulador	17
3.3 El motor de joc	20
4 Entorns d'Aprenentatge per Reforç	23
4.1 Arquitectura en capes	23
4.2 Espai d'observació	24
4.3 Espai d'accions i màscara	26
4.4 Reward shaping	26
5 Metodologia d'Entrenament	29
5.1 Marc metodològic comú	29

5.2	Bloc 1: línia base i <i>curriculum learning</i>	32
5.2.1	Fase 1: comparativa homogènia d'algorismes	32
5.2.2	Fase 2: <i>curriculum learning</i> (mà → partida)	34
5.2.3	Checkpoint 1: models seleccionats	37
5.3	Bloc 2: representació de l'estat i memòria	38
5.3.1	Fase 3: el feature extractor COS	38
5.3.1.1	Arquitectura: CNN vs MLP	38
5.3.1.2	Inicialització supervisada del cos	40
5.3.2	Fase 4: memòria d'oponent (LSTM) i pool d'oponents	42
5.3.3	Checkpoint 2: model seleccionat	44
6	Conclusions	45
A	Hiperparàmetres dels models	47
B	Resultats complets per fase	49
	Bibliografia	51

ÍNDIX DE FIGURES

2.1	Cicle d'interacció agent-entorn en AR.	6
2.2	Tres arquitectures d'aproximació de la funció de valor (VFA): estimació de V , estimació de Q per a un únic parell (s, a) , i estimació simultània de Q per a totes les accions.	7
2.3	Model gràfic del POMDP: els estats s_t (cercles continus) són ocults; l'agent només observa o_t (cercles discontinus) generats per la funció d'emissió O	8
2.4	Cicle d'entrenament DQN (ordre d'execució: 1 observació, 2 acció ϵ -greedy, 3 emmagatzematge, 4 mostreig i càlcul de la pèrdua, 5 actualització de θ , 6 sincronització de θ^- cada N passos).	11
2.5	Arquitectura Actor-Critic de PPO: extractor de característiques compartit amb dos caps independents, un per a la política i un per a la funció de valor.	11
2.6	NFSP: el bucle RL genera accions que s'acumulen al reservoir buffer; el bucle SL aprèn per imitació la política promig $\bar{\pi}$, que s'usa com a oponent durant l'entrenament.	13
3.1	Arquitectura MVC: el controlador només coneix les interfícies.	18
3.2	Fases d'un torn. Sense senyes, la Fase 0 se salta. Quan es canta truc o envit, el motor entra en estat PENDING i el torn passa a l'adversari fins que respon; resoluda l'aposta, el joc reprèn la Fase 1.	21
4.1	Capes de l'entorn. TrucEnvMa i TrucGymEnvMa segueixen la mateixa estructura però amb episodi = 1 mà.	23
5.1	Mètrica composta de la Fase 1 sota les dues òptiques d'avaluació. A l'esquerra, rendiment als 5 M passos d'entrenament (eficiència per mostra); a la dreta, rendiment al llarg de 4 hores de còmput per agent (rendiment efectiu). DQN-SB3 lidera en ambdues, però el rànquing de la resta canvia: PPO-SB3 és el pitjor per passos però supera DQN-RLCard i NFSP-RLCard per temps gràcies al seu <i>throughput</i>	34
5.2	Curriculum de dues etapes: mans individuals amb recompensa densa, se-guides de partides senceres amb transferència dels pesos apresos (θ).	35
5.3	Corbes del <i>curriculum</i> per als agents RLCard. La línia ratllada és la fase de mans (pre-entrenament); la línia sòlida el <i>finetune</i> sobre partides. La línia puntejada marca la transició als 12M passos. DQN-RLCard col·lapsa al <i>finetune</i> ; NFSP-RLCard recupera i millora lleugerament.	36

5.4	Corbes del <i>curriculum</i> per als agents SB3. DQN-SB3 assoleix un pic alt en mans (86%) però perd rendiment al <i>finetune</i> . PPO-SB3 manté el nivell i tanca a 75%, el millor resultat del treball fins al moment.	37
5.5	CosMultiInput: arquitectura de dues branques. La branca CNN processa el tensor de cartes (6,4,9); la branca lineal el vector de context (24,). Les sortides es concatenen i es projecten a un vector latent de 256 dimensions.	39
5.6	Pic de <i>metric</i> (24M passos) per protocol i agent, COS amb pesos aleatoris vs Multilayer Perceptron (MLP). La línia discontinua marca el 80%. El COS només supera clarament el MLP al control de DQN-SB3; al protocol de mans (el millor en absolut) la diferència és marginal.	40
5.7	Esquema del preentrenament supervisat: els tres caps propaguen gradient al cos i l'obliguen a aprendre representacions sensibles a l'estructura del joc. Un cop acabat, els caps es descarten i es conserven només els pesos del cos.	41

ÍNDIX DE TAULES

2.1	Components de la tupla MDP.	6
2.2	Comparativa dels algorismes d'AR emprats al treball.	13
3.1	Jerarquia de força de les cartes, de major a menor.	16
3.2	Escala d'apostes de l'Envit i punts cedits en cas de retirada.	17
3.3	Escala d'apostes del Truc.	17
3.4	Mètodes principals del contracte Model.	18
3.5	Mètodes principals del contracte Vista.	19
4.1	Els sis canals del tensor de cartes obs_cartes.	25
4.2	Les 24 dimensions del vector de context obs_context.	25
5.1	Les sis variants de l'AgentRegles que formen el <i>pool</i> d'oponents (Fases 4–6). La variant equilibrada (primera fila) reproduïx el comportament per defecte usat a les fases 1–3.	31
5.2	Els quatre algorismes comparats a la Fase 1.	32
5.3	Distribució d'oponents per episodi a la Fase 1. Els agents de RLCard incorporen self-play; els de Stable-Baselines3 (SB3) entrenen només contra oponents fixos (RandomAgent i AgentRegles).	33
5.4	metric de la Fase 1 sota les dues òptiques. «Millor» és el pic durant l'entrenament; «Final» és el valor en acabar, que evidencia la inestabilitat.	33
5.5	Throughput dels quatre algorismes. El paral·lelisme de PPO-SB3 el fa 28 vegades més ràpid que NFSP-RLCard.	34
5.6	Comparació a 12M passos (mateix pressupost, avaluació sobre partides). El valor metric entrenant sobre mans és uniformement superior.	36
5.7	Comparació a 24M passos totals (control sencer vs curriculum complet). Només PPO-SB3 i NFSP-RLCard milloren; DQN-RLCard col·lapsa per oblit catastròfic.	36
5.8	Pic de metric (24M passos) per a COS amb pesos aleatoris vs MLP.	40
5.9	Caps de predicció del ModelPreEntrenament per al preentrenament supervisat del cos CosMultiInput.	41
5.10	Pic de metric (protocol mans, 24M passos) per als quatre règims d'inicialització del cos.	42
5.11	Resultats de la Fase 4 (pressupost igualat a 12M passos). DQN frozen mostra el seu màxim dins el pressupost de 12M; el seu pic global és del 91,2% a 23M.	43

LLISTA D'ACRÒNIMS

AR	Aprenentatge per Reforç
CNN	Convolutional Neural Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
DRQN	Deep Recurrent Q-Network
GRU	Gated Recurrent Unit
IA	Intel·ligència Artificial
LSTM	Long Short-Term Memory
MDP	Markov Decision Process
MLP	Multilayer Perceptron
MVC	Model-Vista-Controlador
NFSP	Neural Fictitious Self-Play
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
SB3	Stable-Baselines3
SL	Supervised Learning
TFG	Treball Final de Grau

RESUM

[Resum de 250–500 paraules: problema, mètode, resultats, conclusions.]

Paraules clau: *reinforcement learning, truc, jocs de cartes, DQN, PPO, NFSP, informació parcial, curriculum learning, self-play.*

INTRODUCCIÓ

El joc és una de les activitats més antigues de la humanitat. Més que un objecte d'estudi, és una pràctica social: un espai on les persones s'enfronten, es coordinen, negocien i, sovint, s'ensenyen mútuament. Els antropòlegs el descriuen com un mecanisme de transmissió cultural i d'entrenament de la intel·ligència, present en totes les societats conegudes [1]. Jugar no és només passar l'estona: és **practicar la presa de decisions sota incertesa, gestionar el risc i llegir les intencions dels altres**.

Els jocs tradicionals hi afegeixen una capa de patrimoni i cultura. Aquests no són simples entreteniments: són codis culturals transmesos de generació en generació, amb vocabulari propi, rituals i regles que reflecteixen el pensament social de cada territori. El problema és que necessiten jugadors que coincideixin al mateix lloc i al mateix moment. La dispersió geogràfica, l'envelliment dels cercles de jugadors i el canvi d'hàbits d'oci fan que sigui cada cop més difícil. Sense adversaris ni parella, **el joc s'extingeix per inanició, no per desinterès**.

No record exactament quan vaig aprendre a jugar a cartes. Als tretze o catorze anys ja hi jugaven tots els meus amics del poble. El que sé és que he tengut la sort de poder comptar amb els padrins, ties i altra gent, que m'han pogut ensenyar els distints jocs tradicionals, entre ells jocs de cartes. Al meu poble, jocs com el Terceti (que avui quasi ja no es juga a cap altre lloc) o el Truc formen part de la vida quotidiana, transmesos de generació en generació sense que ningú ho pensés gaire.

Però amb els anys m'he anat adonant que cada cop és més difícil trobar gent per seure a una taula. El cercle de jugadors envelleix, i quan no hi ha prou persones disponibles el Truc simplement no es pot jugar. Aquesta inquietud és el que m'ha duit a fer aquest Treball Final de Grau (TFG): crear una versió del Truc que es pugui jugar contra un agent d'aprenentatge per reforç, de manera que **el joc pugui continuar viu encara que no hi hagi quatre humans disponibles**.

1.1 La intel·ligència artificial en el joc

Darrerament, la relació entre jocs i Intel·ligència Artificial (IA) ha anat més lluny de la preservació cultural. Des de fa dècades, els jocs han servit també com a banc de prova per als investigadors, ja que, en ser entorns tancats amb regles precises i un criteri d'èxit mesurable, permeten avaluar algorismes de manera objectiva i reproducible. L'any 1997, Deep Blue va derrotar el campió mundial d'escacs Garry Kasparov, una fita que semblava inalcançable una dècada abans [2]. El 2016, AlphaGo va superar Lee Sedol al joc del Go, un problema considerat exponencialment més complex pels seus 10^{170} estats possibles [3]. El 2019, l'agent OpenAI Five va derrotar equips professionals de Dota 2 en partides de cinc jugadors per banda [4].

El denominador comú d'aquests avenços és el Deep Reinforcement Learning (DRL): la combinació de l'Aprenentatge per Reforç (AR) clàssic amb xarxes neuronals profundes. L'AR permet a un agent aprendre interactuant amb un entorn i rebent recompenses; les xarxes profundes li permeten representar espais d'estats molt complexos sense necessitat de definir manualment les característiques rellevants. La combinació dels dos ha permès que **agents sense cap coneixement inicial arribin a nivells humans** en diferents dominis (escacs, Go, Dota 2, etc.). Aquest treball segueix el mateix enfocament: xarxes neuronals profundes combinades amb AR per aprendre a jugar al Truc partint de zero.

1.2 El repte de la informació parcial

Tots aquests èxits, però, es produeixen en jocs d'*informació completa*: tots els jugadors veuen l'estat sencer del joc en tot moment. Els escacs, el Go i els jocs de taulell en general pertanyen a aquesta categoria. En canvi, molts dels jocs que els humans troben més fascinants, i estratègicament rics, amaguen informació: les cartes dels adversaris, les intencions futures, els recursos ocults.

Formalment, un joc d'informació parcial es modela com un Partially Observable Markov Decision Process (POMDP): l'agent rep observacions parcials de l'estat real del món i ha d'actuar sense visibilitat completa. L'estratègia òptima ja no consisteix a calcular la millor jugada donada la posició, sinó a **raonar sobre els estats ocults i el possible comportament d'un adversari**.

Els jocs de pòquer han estat el principal camp d'experimentació en informació parcial. El 2017, Libratus va guanyar jugadors professionals de Texas Hold'em de dos jugadors (*heads-up*) [5]. El 2019, Pluribus va estendre aquest resultat al pòquer de sis jugadors, un entorn multiagent molt més complex on les tècniques clàssiques de teoria de jocs no escalen directament [6]. Ambdós agents aproximen un *equilibri de Nash*: una estratègia tal que cap jugador pot millorar el seu resultat canviant unilateralment de política.

Malgrat aquests progressos, el camp és lluny d'estar tancat. Cada joc d'informació parcial té la seva pròpia estructura d'observació, el seu espai d'accions i les seves dinàmiques estratègiques. Un agent entrenat al pòquer no es pot transferir directament a un altre joc de cartes: cal dissenyar l'entorn, definir l'espai d'estats i triar els algorismes adequats a cada cas.

1.3 El Truc: engany, parella i farol

El Truc és un joc de cartes tradicional d'arrel catalana i valenciana, arrelat a les Illes Balears i al País Valencià [7]. Es juga per parelles (dos contra dos) amb una baralla espanyola de 36 cartes, i guanya el primer equip que arriba als 24 punts d'una *cama*. Cada mà es disputa en dues apostes independents: l'**Envit** (sobre la força de la mà en punts) i el **Truc** (sobre qui guanya les rondes de carta alta). Les regles completes, la jerarquia de cartes i el model formal es detallen al capítol 3.

La mecànica no és el que fa del Truc un repte per a la IA. És el **farol**: qui té cartes dolentes pot cantar Truc amb convicció i guanyar el punt sense ensenyar-les. A més, les parelles es coordinen mitjançant *senyes*, un sistema de gestos facials codificats que els adversaris poden intentar llegir i explotar. Com es diu al joc: *"El Truc no és només matemàtiques i memòria. És un joc de cares, de silencis i de mentides."*

Tot plegat, el Truc és un POMDP cooperatiu-competitiu: l'agent ha de gestionar informació oculta, coordinar-se sense comunicació verbal explícita i decidir a cada torn si enganyar o ser sincer sobre la força de la seva mà. Aquesta combinació de factors el fa substancialment diferent dels jocs estudiats fins ara en la literatura.

1.4 Objectius i estructura d'aquest treball

L'objectiu d'aquest TFG és desenvolupar i avaluar agents d'AR capaços de jugar al Truc de manera competitiva, partint de zero i sense cap coneixement previ del joc. L'entrenament dels agents es circumscriu a la variant d'1 vs 1 sense senyes; l'extensió a agents cooperatius per parelles s'identifica (2 vs 2 amb senyes) com a línia de recerca futura. Per assolir aquest objectiu, el treball es desglossa en tres subobjectius:

1. **Modelització, implementació i aplicació jugable.** Dissenyar i implementar un simulador del Truc que implementi totes les regles del joc (envit, truc i apostes), oferir un mode de joc jugador contra agent, i exposar el simulador com a entorn estàndard compatible amb les biblioteques d'AR existents. El motor admet també la variant 2 vs 2 amb senyes, tot i que l'entrenament d'agents es limita a 1 vs 1.
2. **Comparació d'algorismes i tècniques d'entrenament.** Avaluar quatre algorismes (Deep Q-Network (DQN) i Neural Fictitious Self-Play (NFSP) via RLCard, i DQN i Proximal Policy Optimization (PPO) via SB3) en condicions homogènies, i estudiar l'efecte del *curriculum learning* (entrenar primer en mans individuals i després en partides senceres), de l'ús d'extractors de característiques convolucionals, de la memòria recurrent (GRU) i del *self-play* sobre la convergència i el rendiment dels agents.
3. **Avaluació.** Mesurar el rendiment de cada agent contra un agent de regles heurístiques i analitzar si els agents aprenen comportaments estratègics (gestió de l'envit, ús del farol al truc) o es limiten a polítiques reactives.

El treball s'organitza de la manera següent. El capítol 2 presenta el marc teòric: els fonaments de l'AR, els algorismes emprats i l'estat de l'art en jocs d'informació parcial. El capítol 3 descriu la lògica del joc i l'arquitectura del simulador. El capítol 4 detalla el disseny dels entorns d'AR. El capítol 5 explica la metodologia d'aprenentatge per reforç

1. INTRODUCCIÓ

i les fases d'entrenament. El capítol ?? presenta i discuteix els resultats experimentals. Finalment, el capítol ?? recull les conclusions i les línies de treball futur.

FONAMENTS TEÒRICS I TREBALL RELACIONAT

Aquest capítol estableix el marc conceptual del treball en tres blocs. Primer presenta els fonaments de l'AR. A continuació revisa l'estat de l'art en jocs d'informació parcial, que justifica l'elecció d'un enfocament basat en DRL. Finalment descriu els algorismes i tècniques concrets que s'utilitzen a la part experimental.

2.1 Aprenentatge per Reforç

L'AR és el paradigma d'aprenentatge automàtic en el qual un agent aprèn a actuar en un entorn mitjançant la interacció directa, guiat per senyals de recompensa [8]. A diferència de l'aprenentatge supervisat, no existeix cap conjunt d'exemples etiquetats ni un tercer que indiqui la resposta correcta; a diferència de l'aprenentatge no supervisat, sí que hi ha una senyal d'avaluació (la recompensa) que orienta l'aprenentatge.

Aquesta secció presenta els fonaments formals de l'AR: la formalització com a MDP, les funcions de valor i les equacions de Bellman, el dilema exploració-explotació, la distinció entre algorismes *on-policy* i *off-policy*, l'extensió al cas d'observació parcial (POMDP) i el concepte d'equilibri de Nash en jocs multi-agent.

Procés de Decisió de Markov

L'entorn es formalitza com un Markov Decision Process (MDP), definit per la tupla (S, A, P, R, γ) on cada component té el significat que recull la Taula 2.1.

Component	Descripció
S	Conjunt d'estats de l'entorn
A	Conjunt d'accions disponibles per a l'agent
$P(s' s, a)$	Probabilitat de transitar a s' prenent l'acció a des de s
$R(s, a, s')$	Recompensa immediata esperada en la transició
$\gamma \in [0, 1)$	Factor de descompte (recompenses futures vs. immediates)

Taula 2.1: Components de la tupla MDP.

El nom d'*Markov* fa referència a la **propietat de Markov**: l'estat s_t conté tota la informació necessària per prendre la decisió òptima, de manera que el futur és independent del passat donat el present:

$$P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_0, a_0, \dots, s_t, a_t).$$

Això implica que l'agent no necessita recordar els estats anteriors: n'hi ha prou amb l'estat actual per calcular la millor acció.

Una *política* π és la regla de decisió de l'agent: en la forma estocàstica, $\pi(a | s)$ expressa la probabilitat de seleccionar l'acció a quan l'estat és s ; en la forma determinista, $\pi : S \rightarrow A$ assigna directament una acció a cada estat. El cicle bàsic d'interacció és el representat a la Figura 2.1: a cada pas de temps t , l'agent observa s_t , tria $a_t \sim \pi(\cdot | s_t)$, rep la recompensa r_t i l'entorn transita a $s_{t+1} \sim P(\cdot | s_t, a_t)$.

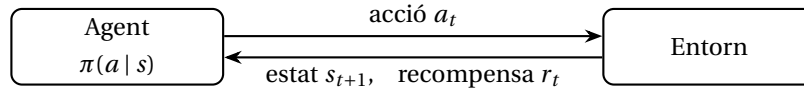


Figura 2.1: Cicle d'interacció agent-entorn en AR.

L'objectiu de l'agent és trobar la política òptima π^* que maximitzi el *retorn acumulat descomptat*:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

El factor γ controla l'horitzó temporal efectiu: quan $\gamma \rightarrow 0$ l'agent és miop i prioritza les recompenses immediates; quan $\gamma \rightarrow 1$ valora el futur quasi tant com el present.

Funcions de valor i equacions de Bellman

Per saber si una política π és bona, cal quantificar el retorn esperat que l'agent obtindrà seguint-la. Per fer-ho s'utilitzen dues funcions de valor complementàries: la *funció de valor d'estat* $V^\pi(s)$ i la *funció de valor d'acció* $Q^\pi(s, a)$:

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s], \quad Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a].$$

Aquestes funcions satisfan les *equacions de Bellman*, que les expressen de forma recursiva a partir del pas immediatament següent:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^\pi(s')], \quad (2.1)$$

$$Q^\pi(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a')]. \quad (2.2)$$

Quan la política és l'òptima π^* , les equacions es simplifiquen substituint la suma sobre accions per un màxim:

$$V^*(s) = \max_a \mathbb{E}[r + \gamma V^*(s') | s, a], \quad (2.3)$$

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a') | s, a]. \quad (2.4)$$

La relació entre totes dues és $V^*(s) = \max_a Q^*(s, a)$, i la política òptima s'obté directament com $\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$.

Aproximació de la funció de valor

Calcular Q^* en la pràctica requereix representar el valor de cada parell (s, a) possible. En entorns petits amb estats discrets n'hi ha prou amb una **taula de consulta** on cada entrada emmagatzema $Q(s, a)$ individualment. El problema és que en la majoria d'entorns reals l'espai d'estats és exponencialment gran: ni s'hi pot emmagatzemar tota la taula en memòria, ni és viable aprendre un valor independent per a cada estat.

La solució és substituir la taula per un **aproximador de funcions** $\hat{Q}(s, a; \mathbf{w})$ parame-
tritzat per un vector de pesos $\mathbf{w}$ que *generalitza* entre estats similars. L'aproximador rep com a entrada l'estat (o el parell estat-acció) i estima el valor Q corresponent. Hi ha tres arquitectures habituals, il·lustrades a la Figura 2.2:

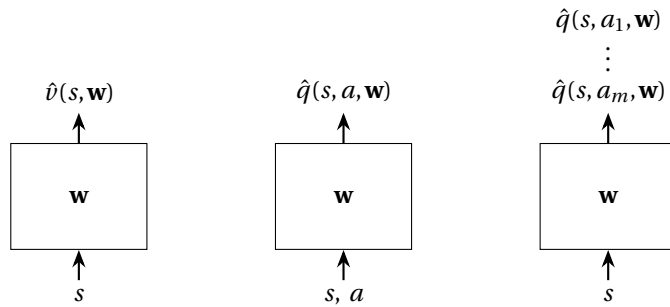


Figura 2.2: Tres arquitectures d'aproximació de la funció de valor (VFA): estimació de V , estimació de Q per a un únic parell (s, a) , i estimació simultània de Q per a totes les accions.

La tercera arquitectura —entrada s , sortida un valor Q per a cada acció possible— és la més eficient per a la presa de decisions, ja que permet comparar totes les accions amb una sola passada per la xarxa. És l'arquitectura que utilitza DQN. L'ús de **xarxes neuronals profundes** com a aproximador és el que dóna nom a la família DRL: l'agent aprèn els pesos $\mathbf{w}$ d'una xarxa $Q(s, a; \mathbf{w})$ que aproxima la funció de valor òptima, en lloc d'una taula.

Exploració i explotació

Fins i tot amb les equacions de Bellman ben definides, aprendre Q^* per interacció directa planteja un dilema pràctic: l'agent ha de decidir si aplica l'acció que creu millor (**explotació**) o en prova una de nova per obtenir més informació (**exploració**). Sense exploració suficient, l'agent queda atrapat en polítiques subòptimes; amb massa exploració, no convergeix.

L'estratègia ϵ -greedy és la solució més senzilla: amb probabilitat $1 - \epsilon$ tria l'acció de valor màxim i amb probabilitat ϵ n'escull una uniformement a l'atzar. Durant l'entrenament, ϵ decau des d'un valor proper a 1 fins a un valor residual baix. Els algorismes de gradient de política com PPO adopten un enfocament diferent: afegeixen un terme d'entropia $\mathcal{H}[\pi]$ a la funció objectiu, penalitzant les polítiques massa deterministes i incentivant la diversitat d'accions de forma contínua.

Polítiques *on-policy* i *off-policy*

Els algorismes *on-policy* aprenen exclusivament de transicions generades per la política que s'està optimitzant en cada moment: les dades queden obsoletes tan bon punt la política s'actualitza. Els algorismes *off-policy*, en canvi, separen la política que genera experiència (*política de comportament*) de la que s'optimitza (*política objectiu*), de manera que les transicions passades poden ser reutilitzades per a múltiples actualitzacions. Aquesta distinció implica compromisos diferent entre estabilitat d'entrenament i eficiència de mostres.

Observació parcial: el model POMDP

Els models MDP vistos fins ara assumeixen que l'agent observa l'estat complet del sistema en cada instant. En molts entorns reals, i en particular al Truc, aquesta suposició no es compleix: els adversaris amaguen les seves cartes i les intencions futures romanen ocultes.

Un POMDP estén el MDP amb dues components addicionals (un conjunt d'observacions Ω i una funció d'emissió O), donant la tupla completa $(S, A, P, R, \gamma, \Omega, O)$. La funció $O(o | s', a)$ especifica la probabilitat de rebre l'observació $o \in \Omega$ després de transitar a l'estat s' prenent l'acció a . L'agent no veu s_t directament, sinó una observació parcial o_t que en conté menys informació, tal com il·lustra la Figura 2.3.

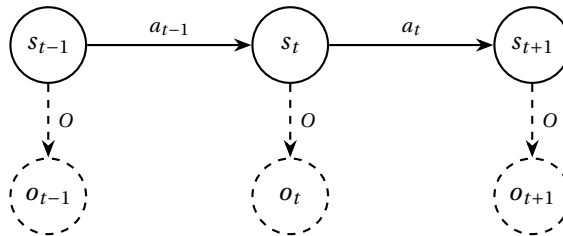


Figura 2.3: Model gràfic del POMDP: els estats s_t (cercles continus) són ocults; l'agent només observa o_t (cercles discontinus) generats per la funció d'emissió O .

La política òptima en un POMDP ha de raonar sobre la *distribució de creença* (*belief state*) $b_t(s) = P(s_t = s \mid o_{1:t}, a_{1:t-1})$. Quan es rep una nova observació o_{t+1} , la creença s'actualitza aplicant el teorema de Bayes:

$$b_{t+1}(s') \propto O(o_{t+1} \mid s', a_t) \sum_{s \in S} P(s' \mid s, a_t) b_t(s).$$

En la pràctica, mantenir b_t explícitament és intractable per a espais d'estats grans. En el seu lloc s'utilitzen xarxes recurrents (LSTM, GRU) que comprimeixen l'historial d'observacions en un vector d'estat ocult, actuant com a representació implícita de la creença.

L'entorn del Truc és precisament un POMDP: l'agent rep una observació que inclou la seva mà i l'historial d'apostes visible, però no les cartes dels adversaris.

Equilibri de Nash

En els jocs multi-agent, el criteri d'optimitat ja no és maximitzar una recompensa contra un entorn fix, sinó respondre bé a un adversari que també optimitza. En un joc de suma zero entre dos jugadors, un **equilibri de Nash** és una parella de polítiques (π^*, σ^*) tal que cap dels dos jugadors pot incrementar el seu retorn esperat canviant unilateralment la seva política:

$$V(\pi^*, \sigma^*) \geq V(\pi, \sigma^*) \quad \forall \pi, \quad V(\pi^*, \sigma^*) \leq V(\pi^*, \sigma) \quad \forall \sigma.$$

Una política en equilibri de Nash és *no explorable*: cap estratègia adversària pot millorar sistemàticament contra ella. Aproximar aquest equilibri és l'objectiu dels mètodes de self-play i NFSP descrits més endavant.

2.2 Estat de l'art en jocs d'informació parcial

El pòquer ha estat el banc de prova principal per a la IA en jocs d'informació parcial. L'estat de l'art actual és la família d'algorismes basats en Minimització del Penediment Contra-factual (*Counterfactual Regret Minimization*, CFR), que han assolit rendiment sobrehumà de forma consistent [5, 6].

Libratus [5] (Brown & Sandholm, 2017) guanya jugadors professionals de Texas Hold'em de dos jugadors. CFR minimitza iterativament el penediment contra-factual (la millora hipotètica si s'hagués pres una acció diferent) i convergeix a l'equilibri de Nash. Libratus resol sub-jocs en temps real per refinar la seva estratègia durant la partida.

Dos anys més tard, els mateixos autors proposaren **Pluribus** [6] (Brown & Sandholm, 2019) estén aquest resultat al pòquer de sis jugadors combinant Monte Carlo CFR, una estratègia de base (*blueprint*) obtinguda per self-play i la resolució de sub-jocs en temps real. Pluribus fou la primera IA a superar humans en un entorn multi-jugador d'informació parcial.

Limitacions dels enfocaments CFR

Malgrat els seus èxits, CFR presenta tres limitacions que motiven l'ús de DRL en aquest treball. El cost creix exponencialment amb la profunditat de l'arbre i el nombre de jugadors, cosa que exigeix abstraccions manuals específiques per a cada joc.

A més, aquestes abstraccions no es transfereixen: redissenyar-les per al Truc (amb apostes menys estructurades i senyes) seria tan costós com el problema original. Finalment, CFR no aprèn representacions compartides, de manera que no es beneficia de la generalització de les xarxes neuronals.

L'enfocament DRL (DQN, PPO, NFSP) evita modelar l'arbre explícitament i generalitza a nous jocs sense redissenyar abstraccions, a canvi d'una convergència a Nash menys garantida teòricament.

2.3 Mètodes usats

Aquesta secció descriu els algorismes i tècniques d'AR que s'empren a la part experimental del treball, des dels algorismes base fins als marcs de self-play.

2.3.1 Deep Q-Network

DQN [9] fou el primer algoritme de DRL a demostrar rendiment comparable al humà en entorns d'alta dimensionalitat. L'agent s'entrenà directament des de fotogrames crus del joc (píxels en escala de grisos), sense cap característica dissenyada manualment, combinant Q-learning amb xarxes neuronals profundes. Va assolir rendiment aproximadament humà en una cinquantena de jocs Atari i sobrehumà en molts d'altres, la qual cosa demostrà que les xarxes neuronals podien aprendre representacions útils per al control directament des de la percepció bruta [9].

En lloc d'una taula $Q(s, a)$, DQN aproxima la funció de valor d'acció amb una xarxa neuronal $Q(s, a; \theta)$ parametritzada per θ . Aprèn minimitzant la pèrdua:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right].$$

Dues innovacions clau estableixen l'entrenament:

- **Experience Replay**: les transicions (s, a, r, s') s'emmagatzemen en un buffer $\mathcal{D}$ i es mostregen aleatòriament. Això trenca la correlació temporal entre mostres consecutives, requisit per a la convergència de la xarxa.
- **Xarxa objectiu θ^-** : una còpia de θ que s'actualitza periòdicament (cada N passes) i s'usa per calcular els objectius de Bellman. Evita l'oscil·lació causada per actualitzar simultàniament la xarxa d'estimació i l'objectiu.

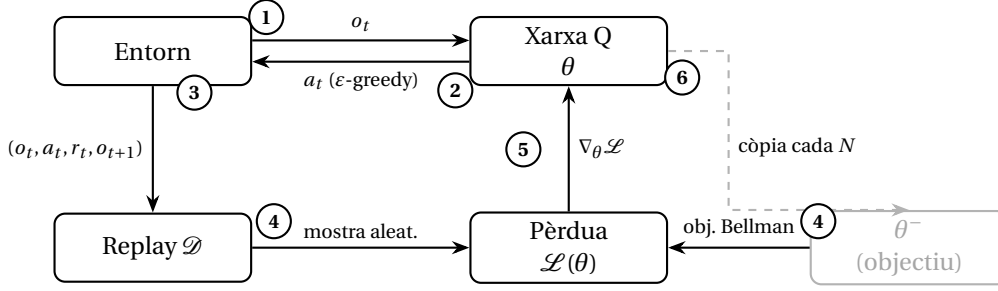


Figura 2.4: Cicle d'entrenament DQN (ordre d'execució: 1 observació, 2 acció ϵ -greedy, 3 emmagatzematge, 4 mostreig i càlcul de la pèrdua, 5 actualització de θ , 6 sincronització de θ^- cada N passos).

2.3.2 Proximal Policy Optimization

PPO [10] és un algoritme Actor-Critic on-policy que s'ha convertit en l'estàndard pràctic de l'AR gràcies a la seva estabilitat i eficiència computacional.

Actor-Critic

PPO manté dues xarxes: l'*actor* $\pi(a | s; \theta)$, que parametriza la política i aprèn a seleccionar la millor acció; i el *crític* $V(s; \phi)$, que parametriza la funció de valor i estima el retorn esperat des de l'estat actual. Ambdues xarxes comparteixen un extractor de característiques comú i divergeixen en dos caps independents (Figura 2.5). L'actor usa la valoració del crític com a referència: si una acció és millor del que el crític esperava, l'actor la reforça; si és pitjor, la penalitza. Aquesta guia redueix la incertesa de l'aprenentatge.

L'avantatge estimat $\hat{A}_t$ mesura si l'acció escollida és millor o pitjor del que preveia el crític, i s'estima mitjançant el mètode GAE (*Generalized Advantage Estimation*).

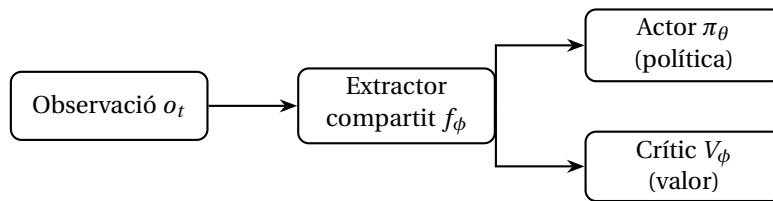


Figura 2.5: Arquitectura Actor-Critic de PPO: extractor de característiques compartit amb dos caps independents, un per a la política i un per a la funció de valor.

Objectiu amb retall

La innovació central de PPO és l'objectiu amb retall (*clipped surrogate objective*), que limita l'actualització de la política per evitar passos massa grans:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)],$$

on $r_t(\theta) = \pi(a_t | s_t; \theta) / \pi(a_t | s_t; \theta_{\text{old}})$ és el ràtio entre la política nova i l'antiga, i $\epsilon = 0,2$.

La pèrdua total afegeix un terme de valor $\mathcal{L}^V$ i un bonus d'entropia $\mathcal{H}[\pi]$ per mantenir l'exploració:

$$\mathcal{L} = \mathcal{L}^{\text{CLIP}} - c_1 \mathcal{L}^V + c_2 \mathcal{H}[\pi].$$

2.3.3 Deep Recurrent Q-Learning

DQN assumeix que l'observació o_t és una representació suficient de l'estat per prendre decisions òptimes. En un POMDP això no és cert: sense memòria de les observacions passades, l'agent pot prendre decisions subòptimes quan l'estat ocult és rellevant.

Deep Recurrent Q-Network (DRQN) [11] proposa substituir la primera capa densa de DQN per una Gated Recurrent Unit (GRU) o LSTM que manté un estat ocult (h_t, c_t) propagat a través dels passos de temps. Així, l'aproximació de la funció de valor depèn no sols de o_t sinó de tota la seqüència d'observacions passades, el que equival a aproximar la funció de valor sobre la *creença* (distribució de probabilitat sobre estats ocults).

L'entrenament propaga els gradients a través del temps (BPTT, *Backpropagation Through Time*) sobre segments de trajectòria de longitud L . La pèrdua s'acumula als últims L passos de la seqüència per evitar que els gradients s'extingeixin en contextos molt llargs.

2.3.4 Neural Fictitious Self-Play

Self-play i Fictitious Play

Aproximar l'equilibri de Nash (definit a la secció 2.1) requereix entrenar contra adversaris cada cop més forts en lloc d'un oponent fix. El self-play aconsegueix precisament això fent que l'agent jugui contra si mateix.

El *Fictitious Play* [12] és un algorisme iteratiu on cada jugador respon amb la millor acció possible a la política promig de l'adversari. Convergeix a l'equilibri de Nash en jocs de suma zero de dos jugadors.

El self-play modern [6] opera de manera similar: l'agent s'entrena jugant contra captures de pesos anteriors (*snapshots*), la qual cosa evita l'especialització excessiva contra un únic oponent i afavoreix la convergència cap a polítiques robustes. La robustesa i l'explotabilitat d'aquestes polítiques es quantifiquen amb mètriques específiques que es defineixen al capítol 5.

De Fictitious Play a NFSP

NFSP [13] duu el Fictitious Play al cas neuronal: és un marc d'entrenament per a jocs d'informació imperfecta que combina l'AR per aprendre la millor resposta (*best response*) amb aprenentatge supervisat per aprendre la política promig, aproximant així l'equilibri de Nash sense resoldre explícitament l'arbre de joc.

Dos bucles en paral·lel

1. **Bucle RL (millor resposta)**: un agent PPO maximitza la recompensa jugant contra el pool d'oponents (que inclou còpies antigues i la política promig). Aquest agent

aprèn una política que respon amb la millor acció possible a les polítiques de l'historial.

2. **Bucle SL (política promig)**: una xarxa auxiliar `AveragePolicyNet` s'entrena per cross-entropy per imitar les accions preses pel bucle RL al llarg de tot l'entrenament. Rep com a etiquetes les accions del bucle RL i aprèn la distribució promig d'accions.

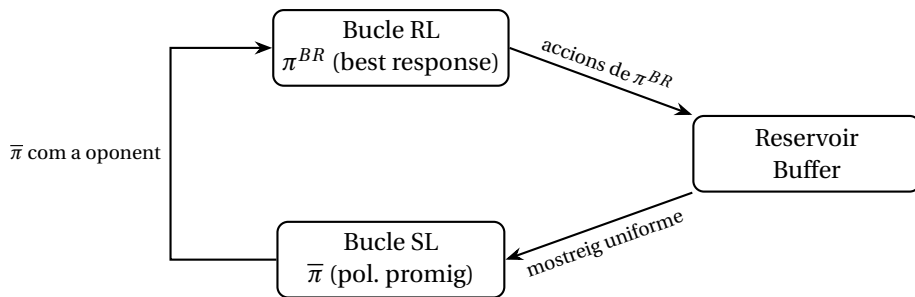


Figura 2.6: NFSP: el bucle RL genera accions que s'acumulen al *Reservoir Buffer*; el bucle SL aprèn per imitació la política promig $\bar{\pi}$, que s'usa com a oponent durant l'entrenament.

Reservoir Buffer

Per garantir que l'estimació de la política promig sigui imparcial, les accions del bucle RL s'emmagatzemen en un *Reservoir Buffer* que aplica mostreig uniforme de l'historial: cada acció passada té la mateixa probabilitat de romandre al buffer, independentment del moment en què es va generar.

Paràmetre η

El paràmetre η controla la probabilitat que l'oponent jugui la política promig (SL) en lloc del pool de self-play durant l'entrenament. La distància a l'equilibri (*Nash gap*) es mesura comparant la millor resposta amb la política promig: si cap de les dues pot guanyar sistemàticament a l'altra, el sistema és a prop de l'equilibri. La mètrica concreta es defineix al capítol 5.

Taula 2.2: Comparativa dels algorismes d'AR emprats al treball.

Algorisme	On/Off-policy	Memòria	Garantia Nash
DQN	Off-policy	Experience replay	No
PPO	On-policy	Cap	No
NFSP	Tots dos	Reservoir buffer	Aproximada

EL JOC DEL TRUC

Abans d'entrenar cap agent cal disposar d'una implementació fidel i verificable del joc. Aquest capítol presenta primer les regles del Truc en la variant considerada, i tot seguit l'arquitectura de programari que les materialitza: un disseny Model-Vista-Controlador (MVC) que separa la lògica del joc, el control del flux i la interfície amb l'usuari. Aquesta separació és la que permet, més endavant, connectar el mateix motor de joc tant a una interfície humana com als entorns d'AR (capítol 4).

3.1 Regles del joc

El Truc és un joc de bases i apostes que es juga per parelles (dos contra dos) o un contra un. L'objectiu és ser el primer equip a assolir els 24 punts d'una *cama*. Es fa servir la baralla espanyola reduïda a 36 cartes: dels quatre colls (Ors, Copes, Espases i Bastos) s'eliminen els 2, 8 i 9, de manera que cada coll conserva nou rangs: {1, 3, 4, 5, 6, 7, 10, 11, 12}.

Jerarquia de les cartes

A diferència d'altres jocs, el valor de combat de les cartes no segueix l'ordre numèric. La taula 3.1 recull la jerarquia completa tal com la implementa el jutge del joc (TrucJudger): les quatre cartes superiors són úniques i reben noms propis, mentre que la resta s'ordena per rang amb desempats entre colls.

3. EL JOC DEL TRUC

Posició	Carta	Denominació
1	11 de Bastos	L'Amo
2	10 d'Ors	Sa Madona
3	As d'Espases	L'espasa
4	As de Bastos	El basto
5	7 d'Espases	Manilla
6	7 d'Ors	Manilla
7	Els tresos	(cap)
8	Assos d'Ors i de Copes	Assos bords
9	Els dotzes	Figures
10	Els onzes	Figures
11	Els deus	Figures
12	7 de Bastos i de Copes	Sets bords
13	Els sisos	
14	Els cincs	Blanques
15	Els quattres	

Taula 3.1: Jerarquia de força de les cartes, de major a menor.

Cada mà es reparteix amb tres cartes per jugador i es disputa en dues apostes independents que es resolen per separat: **l'Envit** i **el Truc**.

L'Envit

L'Envit és l'aposta sobre la força de la mà en punts, i **els jugadors només ho poden cantar a la primera ronda, abans de jugar la seva primera carta**. El valor d'una mà es calcula sumant els valors de les dues cartes més altes d'un mateix coll i afegint-hi 20 punts base, si hi ha dues cartes del mateix coll. A efectes d'aquesta suma, les figures (10, 11 i 12) valen 0, amb dues excepcions: el 10 d'Ors compta 7 i l'11 de Bastos compta 8, i tots dos lliguen amb qualsevol coll. Alguns exemples:

- 7 i 6 d'Espases: $7 + 6 + 20 = 33$ punts.
- As i 5 de Copes: $1 + 5 + 20 = 26$ punts.
- 11 de Bastos i 3 d'Ors: $8 + 3 + 20 = 31$ punts (el 11 de Bastos compta 8 i lliga amb Ors).
- Tres cartes de colls distints (p. ex. 7 d'Ors, 6 de Copes i As de Bastos): no hi ha cap parella del mateix coll, de manera que el jugador no té envit però ho pot cantar igualment com a farol.

Si cap equip canta envit durant la mà, no es disputa cap punt d'envit. Un cop cantat, l'equip contrari pot acceptar, contra-apostar o retirar-se. La retirada cedeix sempre **el valor acumulat fins al moment**: si un equip ha acceptat l'*Envit* (2 pts en joc) i l'adversari canta *Un altre*, retirar-se en aquest punt cedeix 2 punts, no 1 com passaria si rebutgis un *Envit*. Cada crit incrementa els punts en joc segons la taula 3.2. En cas d'empat de

puntuació, guanya l'equip que és **mà** (el jugador que juga immediatament després del qui ha repartit).

Crit	Punts en joc	Si et retires
Envit	2	1 pt
Un altre	4	2 pts (el nivell anterior acceptat)
Dos més	6	4 pts
La Falta	punts que falten al líder	6 pts

Taula 3.2: Escala d'apostes de l'Envit i punts cedits en cas de retirada.

El Truc

El Truc és l'aposta sobre qui guanya les cartes. La mà es resol en fins a tres rondes (anomenades *bassetges*): a cada ronda els jugadors juguen una carta i en guanya la de rang superior segons la taula 3.1; l'equip que s'endú dues bassetges guanya la mà. En qualsevol moment un jugador pot cantar *Truc*, obrint una escala d'apostes independent de l'Envit (taula 3.3). Com en l'Envit, el contrari pot acceptar, contra-apostar o retirar-se cedint el nivell anterior. Els empats es resolen de la manera següent: si la primera ronda acaba en empat, la guanya qui s'imposi a la segona; si és la segona la que empata, puntua qui havia guanyat la primera; si empaten totes dues primeres rondes i la tercera també acaba en empat, guanya l'equip que és **mà**.

Crit	Punts en joc
Truc	3
Retruc	6
Quatre Val	9
Joc Fora	tota la cama

Taula 3.3: Escala d'apostes del Truc.

Les senyes

En la modalitat per parelles, els companys es comuniquen mitjançant *senyes*: un conjunt de nou gestos facials codificats que indiquen quines cartes fortes es tenen, sense parlar i procurant que els rivals no els vegin. Per exemple, alçar les celles indica que es té L'Amo, i aclucar un ull, Sa Madona. La implementació recull aquests nou senyals com a accions del joc, però el seu ús queda restringit a les partides 2 contra 2 entre humans; l'entrenament dels agents es fa sobre la variant 1 contra 1 sense senyes (vegeu el capítol 5).

3.2 Arquitectura del simulador

El simulador segueix el patró MVC, que separa el sistema en tres responsabilitats independents:

- El **model** conté la lògica del joc: l'estat, les regles i els jugadors automàtics.
- La **vista** gestiona l'entrada i la sortida amb l'usuari.
- El **controlador** orquestra el flux, demanant dades i accions a la vista i consultant i modificant l'estat a través del model.

La clau del disseny és que el controlador (`controlador/controlador.py`) no depèn de cap implementació concreta, sinó de dues *interfícies* (contractes): qualsevol vista i qualsevol model que les satisfacin són intercanviables sense tocar el controlador. Aquesta indirecció és la que permet reutilitzar el mateix nucli de joc per a una interfície gràfica humana o per als adaptadors d'AR. La figura 3.1 resumeix aquesta organització.

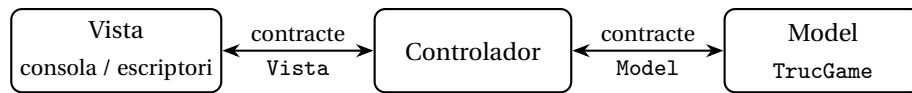


Figura 3.1: Arquitectura MVC: el controlador només coneix les interfícies.

Els contractes

El controlador interactua amb el model mitjançant el protocol `Model` i amb la vista mitjançant el protocol `Vista`, tots dos definits a `controlador/interficie_model.py` i `vista/interficie_vista.py` respectivament. Les taules 3.4 i 3.5 en resumeixen els mètodes principals.

Mètode	Funció
<code>iniciar(config)</code>	Crea i inicialitza una partida
<code>get_estat(pid)</code>	Retorna l'estat visible per a un jugador
<code>get_jugador_actual()</code>	Indica qui ha de jugar ara
<code>es_huma(pid)</code>	Indica si el jugador és humà
<code>get_accio_bot(pid)</code>	Retorna l'acció triada per un bot
<code>aplicar_accio(a)</code>	Aplica una acció i avança l'estat
<code>get_guanyador_envit_recent()</code>	Retorna equip, punts i detall de l'envit tancat
<code>get_guanyador_truc_recent()</code>	Retorna equip i punts del truc tancat
<code>es_final()</code>	Indica si la partida ha acabat
<code>get_resultat()</code>	Retorna el marcador i els <i>payoffs</i> finals

Taula 3.4: Mètodes principals del contracte `Model`.

Mètode	Funció
<code>demanar_config()</code>	Demana la configuració de la partida
<code>mostrar_estat(estat)</code>	Mostra l'estat actual del joc
<code>escollir_accio(legals, estat)</code>	Presenta les accions legals i en retorna una
<code>mostrar_accio(pid, nom, bot)</code>	Informa de l'acció feta (bot afegeix retard visual)
<code>mostrar_guanyador_envit(...)</code>	Comunica qui ha guanyat l'envit de la mà
<code>mostrar_guanyador_truc(...)</code>	Comunica qui ha guanyat el truc
<code>mostrar_fi_partida(...)</code>	Mostra el resultat final
<code>demanar_repetir()</code>	Pregunta si es vol jugar una altra partida
<code>mostrar_sortint()</code>	Indica que l'usuari surt de l'aplicació

Taula 3.5: Mètodes principals del contracte Vista.

El bucle de joc

El controlador (`Controlador.executar_partida`) implementa un bucle únic que avança mentre la partida no acabi. A cada iteració consulta qui té el torn; si és un humà, mostra l'estat per la vista, li demana una acció i l'aplica al model; si és un bot, obté l'acció directament del model i l'aplica. Després de cada jugada comunica a la vista els esdeveniments rellevants (qui ha guanyat l'envit o el truc) i, en acabar, mostra el resultat. Tota la lògica de joc resta al model i tota la presentació resta a la vista; el controlador només coordina.

Listing 3.1: Bucle principal de `Controlador.executar_partida`.

```

1  def executar_partida(self, override_config=None):
2      config = override_config or self.vista.demanar_config
3      ()
4      self.model.iniciar(config)
5
6      while not self.model.es_final():
7          pid = self.model.get_jugador_actual()
8          if self.model.es_huma(pid):
9              estat = self.model.get_estat(pid)
10             self.vista.mostrar_estat(estat)
11             accio = self.vista.escollir_accio(
12                 estat["accions_legals"], estat)
13             self.model.aplicar_accio(accio)
14             self.vista.mostrar_accio(pid, ACTION_LIST[
15                 accio],
16                                     es_bot=False)
17             else:
18                 accio, nom = self.model.get_accio_bot(pid)
19                 self.model.aplicar_accio(accio)
20                 self.vista.mostrar_accio(pid, nom, es_bot=True)
21             )
22             self._comunicar_esdeveniments_recents()
23
24         resultat = self.model.get_resultat()
25         self.vista.mostrar_fi_partida(
26             resultat["score"], resultat["payoffs"])

```

L'adaptador principal del model és `ModelInteractiu`, que embolcalla una instància de `TrucGame` i li afegeix dues capacitats: aïllar el motor de joc darrere del contracte i injectar models d'AR preentrenats als jugadors automàtics, de manera que un bot pugui ser tant un agent aleatori com una xarxa neuronal entrenada.

Les interfícies

El projecte ofereix dues vistes plenament funcionals. La **vista de consola** és una interfície de text, lleugera i adequada per a execucions automàtiques i depuració en entorns sense pantalla. La **vista d'escriptori** és una interfície gràfica que representa l'estat amb imatges reals de naips i recull les accions per clic de ratolí, intercalant retards artificials en els moviments dels bots per emular el ritme d'un rival humà. La interfície gràfica permet jugar partides 1 contra 1 (humà contra humà o humà contra agent) i 2 contra 2 entre humans amb suport de senyes.

3.3 El motor de joc

El cor del model és la classe `TrucGame` (`entorn/game.py`), que gestiona tot l'estat de la partida (jugadors, cartes, apostes, rondes i puntuació) de manera independent de qualsevol interfície. Delega responsabilitats concretes en tres rols especialitzats, cadascun al seu propi fitxer dins `entorn/rols/`: `TrucDealer` barreja i reparteix les cartes; `TrucJugador` resol la jerarquia, els guanyadors de rondes i el càlcul de l'envit; i `TrucPlayer` representa cada jugador, mantenint la mà actual, la mà inicial (necessària per resoldre l'envit encara que les cartes ja s'hagin jugat) i, opcionalment, el model que decideix les seves accions.

Accions i estat de resposta

El conjunt d'accions del joc és fix i consta de 19 accions: tres per jugar cada carta de la mà, set per a apostes i respostes, i nou per a les senyes. Les accions d'aposta es divideixen per joc: per a l'Envit, `apostar_envit`, `vull_envit` i `fora_envit`; per al Truc, `apostar_truc`, `vull_truc`, `fora_truc` i `passar`. Acceptació i rebuig existeixen per separat: `fora_envit` no clou la mà (el joc continua sense puntuar l'envit), mentre que `fora_truc` implica retirar-se i cedeix els punts en joc. En cada torn, el motor restringeix les accions legals segons la situació.

Un mecanisme central és l'*estat de resposta* (enum `ResponseState`), que pot prendre tres valors: `NO_PENDING`, `TRUC_PENDING` i `ENVIT_PENDING`. Mentre hi ha una aposta pendent, l'adversari només pot acceptar-la, contra-apostar-la o retirar-se: no pot jugar cartes ni fer cap altra acció fins que l'aposta es resolgui. De manera anàloga, quan l'envit queda resolt els punts no s'afegeixen immediatament al marcador sinó que queden suspesos fins que el truc de la mà es tanca, de manera que el marcador reflecteix sempre una mà completament decidida.

Les fases del torn

Per eficiència en l'entrenament, l'estat d'un torn s'ha reduït a dues fases (figura 3.2). La **Fase 0** (senyals) només existeix quan les senyes estan actives i permet registrar un

gest abans de jugar; si estan desactivades, se salta completament. La **Fase 1** unifica apostes i joc de cartes: és on es canta l'envit o el truc, es responen les apostes i es juguen les cartes.

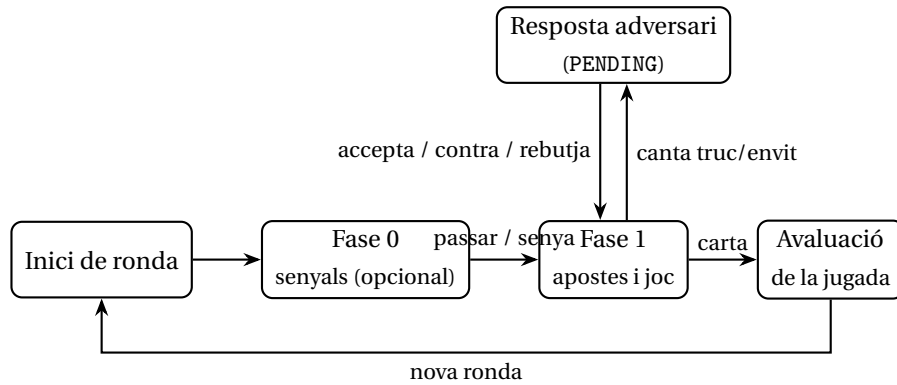


Figura 3.2: Fases d'un torn. Sense senyals, la Fase 0 se salta. Quan es canta truc o envit, el motor entra en estat PENDING i el torn passa a l'adversari fins que respon; resoluda l'aposta, el joc reprèn la Fase 1.

Fi de mà i fi de partida

El motor distingeix dos nivells de finalització, tots dos implementats a `entorn/game.py`. La funció `is_ma_over` indica que la mà actual ha acabat (perquè hi ha un guanyador per rondes o perquè algú s'ha retirat), moment en què es reparteixen els punts i es reparteix una mà nova. La funció `is_over` indica que la partida sencera ha acabat, quan un equip arriba a la puntuació final de la cama.

A banda del resultat final de la partida, el motor emet senyals de recompensa intermèdia (`reward_intermedis`) després de cada jugada rellevant, que s'inclouen dins l'estat retornat a cada pas. El disseny detallat d'aquestes recompenses es descriu al capítol 4.

ENTORNS D'APRENENTATGE PER REFORÇ

Per entrenar agents d'AR al Truc cal adaptar el motor de joc a la interfície estàndard que esperen les biblioteques d'AR. Aquesta adaptació no és trivial: el motor gestiona el torn de múltiples jugadors, manté un estat intern ric i emet senyals de recompensa que les biblioteques genèriques no entenen de manera nativa. Aquest capítol descriu com s'ha organitzat aquesta adaptació en capes, quines decisions s'han pres per representar l'observació i les accions, i com s'ha dissenyat el sistema de recompenses.

4.1 Arquitectura en capes

L'entorn s'organitza en dos nivells d'abstracció superposats (figura 4.1):

1. **Capa RLCard** (TrucEnv / TrucEnvMa): hereta de la classe base Env de RLCard i encapsula el bucle de joc. S'encarrega de convertir l'estat intern del motor en observacions estructurades (mètode `_extract_state`), d'exposar les accions legals en cada pas (`_get_legal_actions`) i de recollir les trajectòries completes.
2. **Capa Gymnasium** (TrucGymEnv / TrucGymEnvMa): embolcalla la capa RLCard amb la interfície estàndard de Gymnasium (mètodes `reset` i `step`) per fer-la compatible amb Stable-Baselines3 (SB3). Gestiona els torns de l'oponent (`_skip_opponent_turns`) i acumula els rewards intermedis que es generen entre dues accions consecutives de l'agent aprenent.

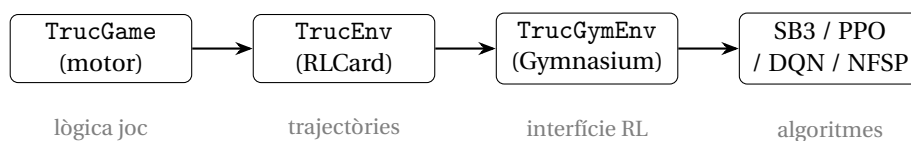


Figura 4.1: Capes de l'entorn. TrucEnvMa i TrucGymEnvMa segueixen la mateixa estructura però amb episodi = 1 mà.

La distinció entre `TrucEnv` i `TrucEnvMa` rau en la granularitat de l'episodi: `TrucEnv` fa córrer una partida completa fins als 24 punts (centenars de passos), mentre que `TrucEnvMa` atura l'episodi quan acaba la mà actual (entre 6 i 20 passos aproximadament). Aquesta variant de gra fi és clau per al *curriculum learning*: l'agent aprèn primer a gestionar mans individuals i posteriorment s'enfronta a partides senceres. El detall del curriculum es descriu al capítol 5.

A més d'aquests dos parells, hi ha dues variants especialitzades:

- `TrucGymEnvSessio` (`joc/entorn_ma/gym_env_sessio.py`): l'episodi agrupa N mans consecutives contra el mateix oponent (per defecte $N = 5$). L'oponent se sampleja del *pool* a cada `reset()`, cosa que obliga l'agent a acumular context entre mans i potencialment detectar els patrons de l'oponent. S'ha usat a la Fase 4 (memòria cross-mans, DRQN).
- `parallel_env_ma.py`: procés fill (*worker*) que executa `TrucEnvMa` en un procés separat via `multiprocessing` i retorna les trajectòries amb els rewards intermedis al procés principal. Permet col·lectar dades de múltiples episodis en paral·lel durant l'entrenament propi de NFSP.

El mètode `set_opponent()` de `TrucGymEnv` permet canviar l'agent oponent en calent sense reconstruir l'entorn; això és imprescindible per als callbacks d'entrenament que implementen el pool d'oponents i el curriculum.

4.2 Espai d'observació

L'observació que rep l'agent en cada pas és un diccionari amb dues entrades complementàries, generades per `_extract_state` a `env.py`:

- `obs_cartes`: tensor de forma (6, 4, 9), que codifica en format *one-hot* la presència de cada carta en sis canals semàntics.
- `obs_context`: vector de 24 valors reals que recull informació numèrica de l'estat de la partida.

Junts representen l'estat observable del joc en 240 dimensions. La funció `flatten_obs` és l'única font de veritat per a la conversió a vector pla:

$$\mathbf{o} = [\text{obs_cartes.flatten()} \parallel \text{obs_context}] \in \mathbb{R}^{216+24} = \mathbb{R}^{240} \quad (4.1)$$

Qualsevol codi que necessiti el vector pla (algoritmes SB3, NFSP, avaluació) crida aquesta funció per garantir coherència.

Tensor de cartes (6, 4, 9)

Les dimensions del tensor corresponen a: eix 0 = canal semàntic (6), eix 1 = pal (4: Espases, Copes, Ors, Bastos), eix 2 = rang (9: {1, 3, 4, 5, 6, 7, 10, 11, 12}). Cada cel·la val 1 si la carta corresponent hi és present i 0 altrament.

Canal	Contingut	Descripció
0	Mà actual	Cartes que té l'agent a la mà
1	Historial propi	Cartes que ja ha jugat l'agent
2	Historial Rival 1	Cartes jugades pel rival de la dreta
3	Historial Company	Cartes jugades pel company (2v2)
4	Historial Rival 2	Cartes jugades pel segon rival (2v2)
5	Senyes company	Cartes revelades per les senyes del company

Taula 4.1: Els sis canals del tensor de cartes `obs_cartes`.

Els índexs de canal 1–4 s'assignen de manera relativa al jugador actual: l'offset k correspon al jugador $(id + k) \bmod n$, de manera que la mateixa estructura de tensor és vàlida per a qualsevol posició de la taula i per a les modalitats 2v2 i 1v1.

Vector de context (24 dimensions)

Índex(os)	Significat	Rang	Codificació
0	Puntuació equip propi	$[0, 1]$	$p_{\text{propi}}/24$
1	Puntuació equip rival	$[0, 1]$	$p_{\text{rival}}/24$
2	Nivell aposta de Truc	$[0, 1]$	$\text{level}_{\text{truc}}/24$
3	Nivell aposta d'Envit	$[0, 1]$	$\text{level}_{\text{envit}}/24$
4	Fase del torn	$\{0, 1\}$	0 = senyes, 1 = apostes/joc
5	Ronda actual	$[0, 1]$	$\text{ronda}/\text{cartes_jugador}$
6–9	Qui és <i>mà</i>	$\{0, 1\}^4$	One-hot relatiu a l'agent
10–13	Qui ha cantat Truc	$\{0, 1\}^4$	One-hot relatiu; 0 si no s'ha cantat
14–16	Qui ha cantat Envit	$\{0, 1\}^3$	One-hot relatiu; 0 si no s'ha cantat
17	Rondes guanyades (propi)	$[0, 1]$	$r_{\text{propi}}/3$
18	Rondes guanyades (rival)	$[0, 1]$	$r_{\text{rival}}/3$
19	Guanyador ronda 1	$\{-1, 0, 1\}$	+1 = propi, -1 = rival, 0 = empat
20	Guanyador ronda 2	$\{-1, 0, 1\}$	+1 = propi, -1 = rival, 0 = empat
21	Envit acceptat	$\{0, 1\}$	Binari
22	Truc pendent	$\{0, 1\}$	1 si TRUC_PENDING
23	Envit pendent	$\{0, 1\}$	1 si ENVIT_PENDING

Taula 4.2: Les 24 dimensions del vector de context `obs_context`.

Totes les variables s'expressen de manera relativa a la perspectiva de l'agent actual. Això fa que el tensor sigui invariant a la posició de la taula: no cal cap transformació addicional per canviar de jugador.

Aquest vector de 240 dimensions és l'entrada de la xarxa de l'agent. L'arquitectura concreta de l'extractor de característiques que el processa (en particular la comparació entre un Multilayer Perceptron (MLP) pla i un extractor convolucional que respecta l'estructura $6 \times 4 \times 9$ del tensor de cartes) es tracta al capítol 5.

4.3 Espai d'accions i màscara

L'espai d'accions conté 19 accions discreta, definides al capítol 3. No totes elles són legals en tot moment: si hi ha un Truc pendent, l'agent només pot respondre (vull, fora o contra); si s'estan jugant cartes no pot apostar si ja ha apostat abans en la mateixa mà, etc.

El mètode `_get_legal_actions()` retorna un `OrderedDict` amb les accions vàlides en el pas actual. En els wrappers `Gymnasium`, aquest conjunt s'emmagatzema a `self._legal_actions` i s'usa de dues maneres:

- **Fallback d'acció il·legal:** si el model prediu una acció fora del conjunt legal, `TrucGymEnv.step()` la substitueix per la primera acció legal (`legal_actions[0]`). Això garanteix que l'entorn mai no genera errors per accions invàlides durant les primeres fases d'entrenament, quan el model és quasi aleatori.
- **Màscara explícita:** en algorismes que ho suporten (com el PPO amb màscares de SB3), es pot passar la llista de booleà directament al model per forçar que la distribució de política sigui zero en accions il·legals.

En jocs amb apostes encadenades com el Truc, la màscara és especialment important: sense ella, l'agent podria intentar re-apostar quan ja no és possible, o respondre a una aposta que ja ha estat resolta. La separació entre les accions definides al motor i les legals en cada pas permet que el motor mantingui la lògica de negoci sense barrejar-la amb la política de l'agent.

4.4 Reward shaping

En una partida fins a 24 punts, la recompensa final pot trigar centenars de passos a arribar. Sense senyals intermèdies, l'agent sabria si ha guanyat o perdut però no podria atribuir el resultat a cap acció concreta: el problema d'assignació de crèdit fa que aprendre a gestionar l'envit o a forçar el truc sigui pràcticament impossible en condicions de recompensa escassa. La solució és el *reward shaping*.

Recompensa terminal

`get_payoffs()` retorna la recompensa final quan la partida (o la mà, en el cas de `TrucEnvMa`) acaba:

- **TrucEnv (partida completa):** +1,0 per a l'equip guanyador, -1,0 per al perdedor.
- **TrucEnvMa (mà individual):** $\pm \text{punts}/24$, on *punts* són els punts de la mà (Truc + Envit). Normalitzar per 24 manté la recompensa a l'escala de la partida completa.

Recompenses intermèdies

El motor emmagatzema les recompenses intermèdies al camp `reward_intermedis` de cada transició. Atès que `RLCard` posa per disseny recompensa zero a totes les transicions intermèdies, la funció `reorganize_amb_rewards` postprocessa les trajectòries

un cop acabat l'episodi: per a cada transició extreu el camp `reward_intermedis` del `raw_obs` i substitueix el zero pel valor acumulat de l'equip, deixant intacte el *payoff* final.

Als wrappers `Gymnasium`, `_reward_from_raw()` extreu directament el reward intermedi en cada crida a `step()` i l'amplifica per un factor 5:

$$r_t = \text{reward_intermedis}[\text{equip}] \times 5,0 \quad (4.2)$$

El factor 5 equilibra l'escala entre els senyals intermedis (ordre de 0.1–0.5) i la recompensa terminal (± 1.0), donant suficient pes al feedback immediat sense eclipsar l'objectiu final.

Les recompenses intermèdies que emet el motor es desglossen de la manera següent:

- **Guanyar una ronda:** $+0,40 \times w_r$ per al guanyador i $-0,20 \times w_r$ per al perdedor, on w_r és un pes decreixent per ronda ($w_1 = 2,0$, $w_2 = 1,0$, $w_3 = 0,5$) que reflecteix que les primeres rondes són més determinants. Si perdre la ronda implica perdre la mà, la penalització s'amplifica per un factor 1,5.
- **Envit acceptat:** recompensa proporcional als punts en joc per a l'equip que matemàticament guanya l'envit.
- **Envit rebutjat (fora_envit):** $+1,5 \times$ per a l'equip que força el rebuig, $-1,0 \times$ per al que es retira.
- **Truc rebutjat (fora_truc en resposta):** $+1,5 \times$ per a l'agressiu, $-1,0 \times$ per al retirat.
- **Retirada voluntària (fora_truc sense resposta pendent):** $+1,0 \times$ per a l'oponent, $-1,2 \times$ per al que fuig, per penalitzar les fugides injustificades.
- **Guanyar la mà (truc net):** $\pm 1,0 \times$ el factor base.

La variant per mans (`TrucGameMa`) simplifica el sistema: acumula un únic reward net al final de cada mà com $\pm$ punts/24, sense senyals de ronda individuals. Això afavoreix un aprenentatge més estable per a la fase de *curriculum learning* on cada episodi és una única mà.

METODOLOGIA D'ENTRENAMENT

Aquest capítol descriu el procés experimental que ha portat des d'agents incapaços de batre una heurística senzilla fins a un agent competitiu. L'enfocament és incremental: cada fase planteja una hipòtesi concreta, dissenya un experiment per validar-la, n'analitza els resultats i pren una decisió que condiciona la fase següent. Aquesta estructura (*hipòtesi* → *disseny* → *resultats* → *decisió*) permet aïllar l'efecte de cada tècnica i justificar per què el treball avança en una direcció i no en una altra.

Les fases s'agrupen en tres blocs, separats per dos punts de control (*checkpoints*) que tanquen un grup d'experiments i fixen quins models continuen:

- **Bloc 1** (secció 5.2): línia base comparativa i *curriculum learning* (Fases 1–2, Checkpoint 1).
- **Bloc 2** (secció 5.3): representació de l'estat i memòria (Fases 3, 3.5 i 4, Checkpoint 2).
- **Bloc 3** (secció ??): robustesa i aproximació a l'equilibri (Fases 5–6).

5.1 Marc metodològic comú

Totes les fases comparteixen el mateix protocol d'avaluació, la mateixa mètrica i el mateix oponent de referència. Aquesta secció els defineix una vegada per no repetir-los a cada fase.

Protocol d'avaluació

El rendiment d'un agent es mesura amb la funció `evaluar_agent()`, que juga un nombre fix de partides senceres contra dos oponents de referència: 100 partides contra un agent aleatori (`RandomAgent`) i 100 partides contra l'agent de regles (`AgentRegles`). Per evitar el biaix posicional (ser *mà* dona un lleuger avantatge), l'agent alterna la seva posició (jugador 0 i jugador 1) en partides consecutives. L'avaluació es fa cada

500 000 passos d'entrenament i sempre sobre **partides senceres**, encara que l'agent s'estigui entrenant sobre mans individuals; així es mesura sempre la competència real al joc complet.

Mètrica de rendiment

S'ha triat el *win-rate* (percentatge de partides guanyades) com a base de la mètrica, en lloc de la recompensa acumulada, per tres motius: és l'objectiu últim del joc, és directament interpretable i és comparable entre algorismes diferents. Aquest darrer punt és el més important. La recompensa d'entrenament de PPO i la de DQN no són directament comparables perquè cada algorisme optimitza una quantitat interna distinta: PPO maximitza un objectiu de gradient de política sobre l'avantatge estimada, mentre que DQN minimitza l'error de diferència temporal de la seva funció Q (secció 2.3.1 i secció 2.3.2). A més, els dos algorismes usen factors de descompte diferents ($\gamma = 0,995$ per a PPO i $\gamma = 0,99$ per a DQN), de manera que ni tan sols el retorn acumulat es mesura a la mateixa escala. El win-rate, en canvi, és una mètrica *externa*: es calcula jugant partides i comptant victòries, amb independència de com s'ha entrenat l'agent, i per això permet comparar qualsevol parella d'algorismes en igualtat de condicions. La mètrica combina el win-rate contra dos oponents complementaris:

$$\text{metric} = 0,25 \cdot \text{wr}_{\text{random}} + 0,75 \cdot \text{wr}_{\text{regles}} \quad (5.1)$$

Cada terme compleix una funció diferent. L'AgentRegles (pes 0,75) és l'oponent **discriminant**: batre una heurística coherent requereix estratègia real, de manera que aquest terme és el que captura la qualitat de l'agent. El RandomAgent (pes 0,25) actua com a **xarxa de seguretat**: guanyar-li és trivial, però un agent que no hi guanyi de manera clara revela un col·lapse de la política (per exemple, una política degenerada que repeteix sempre la mateixa acció). Aquest terme detecta degeneracions que el win-rate contra regles, per si sol, podria emmascarar. El pes dominant recau, doncs, sobre l'oponent que de debò discrimina l'habilitat, mentre que el terme aleatori aporta un sòl d'estabilitat. Les fases avançades (5 i 6) introdueixen mètriques addicionals de robustesa i explotabilitat, que es defineixen al bloc corresponent.

L'oponent de referència: l'agent de regles

L'AgentRegles (RL/models/model_propi/agent_regles.py) és un agent heurístic codificat a mà que serveix alhora com a oponent d'entrenament i d'avaluació. No té cap xarxa neuronal: la seva política prové d'heurístiques amb llindars adaptatius sobre la força de la mà, els punts d'envit i el marcador.

La variant equilibrada i els seus paràmetres. El comportament base de l'agent (la variant *equilibrada*) està governat per quatre paràmetres, cadascun amb un valor neutre que es pren com a referència. Cada paràmetre modula un aspecte de la política:

- `envit_agressio` (neutre 1,0): escala els llindars de punts per acceptar o apujar l'envit. Amb el valor neutre, l'agent apuja l'envit a partir de 30 punts de mà i l'accepta a partir de 25; el paràmetre divideix aquests llindars, de manera que un valor més alt els abaixa (accepta amb mans més febles) i un de més baix els apuja

(més conservador). Els llindars també s'ajusten segons el marcador: si el rival és a punt de guanyar la partida, l'agent es torna conservador; si l'envit pot donar-li la partida, es torna agressiu.

- `truc_agressio` (neutre 1,0): probabilitat d'apostar el truc en situacions favorables (per exemple, havent guanyat una ronda i amb una carta alta). Amb 1,0 aposta sempre en aquests casos; valors menors el fan dubtar i valors majors hi afegeixen farols addicionals.
- `farol_prob` (neutre 0,12): probabilitat d'apostar el truc *sense* una mà forta, és a dir, de fer farol.
- `resposta_truc` (neutre 1,0): disposició a acceptar un truc cantat pel rival. Ajusta el llindar de força exigida per acceptar: valors superiors a 1 accepten amb menys mà, valors inferiors es retiren més sovint.

Estocasticitat. La característica clau de l'agent és que és **estocàstic**: sobre les decisions deterministes anteriors hi injecta soroll aleatori (± 2 punts als llindars d'envit, ± 5 al llindar de força del truc, i decisions probabilístiques a les zones grises de cada llindar). Aquesta estocasticitat és deliberada i essencial. Un oponent purament determinista seria fàcilment explotable: l'agent d'AR aprendria a predir-ne les respostes fixes i el batria de manera artificial, inflant la mètrica sense reflectir cap habilitat real (*reward hacking* contra un oponent fix). El soroll obliga l'agent entrenat a generalitzar i fa de l'AgentRegles una barrera molt més exigent del que sembla.

El pool de variants. A partir de la Fase 4, els quatre paràmetres es modifiquen per generar sis **variants** amb estils de joc contrastats, que formen un *pool* d'oponents (taula 5.1): des de la conservadora (poc agressiva i gairebé sense farol) fins a la farolera (màxima probabilitat de farol), passant per especialistes en truc o en envit. La variant equilibrada hi figura amb els valors neutres descrits més amunt.

Variant	<code>truc_agressio</code>	<code>envit_agressio</code>	<code>farol_prob</code>	<code>resposta_truc</code>
Equilibrat	1,0	1,0	0,12	1,0
Conservador	0,5	0,5	0,02	0,6
Agressiu	1,8	1,8	0,30	1,5
Truc-bot	2,0	0,4	0,15	1,2
Envit-bot	0,4	2,0	0,05	0,7
Faroler	1,3	1,3	0,40	1,3

Taula 5.1: Les sis variants de l'AgentRegles que formen el *pool* d'oponents (Fases 4–6). La variant equilibrada (primera fila) reproduïx el comportament per defecte usat a les fases 1–3.

Convencions comunes

Llevat que s'indiqui el contrari, tots els agents comparteixen l'arquitectura base de xarxa (MLP de dues capes ocultes de 256 unitats amb ReLU) i s'avaluen sobre l'observació de 240 dimensions descrita al capítol 4. El pressupost d'entrenament s'expressa en

timesteps (passos d'interacció amb l'entorn). Els hiperparàmetres complets de cada algorisme i fase es recullen a l'annex A; al cos del capítol només se'n destaquen els rellevants per a cada decisió.

5.2 Bloc 1: línia base i *curriculum learning*

El primer bloc estableix un punt de partida rigorós (Fase 1) i introdueix la primera tècnica que trenca el sostre de rendiment inicial (Fase 2, *curriculum learning*). El Checkpoint 1 tanca el bloc seleccionant els dos models que continuen.

5.2.1 Fase 1: comparativa homogènia d'algorismes

Hipòtesi. Abans d'aplicar cap tècnica avançada cal saber quin algorisme parteix amb avantatge. La Fase 1 compara quatre algorismes d'AR en igualtat de condicions i partint de zero, sense cap coneixement previ del joc.

Disseny. Es comparen quatre agents que cobreixen les dues grans famílies d'algorismes d'AR: els **basats en la funció de valor** (com el DQN, que aprèn quant val cada acció i n'extreu la política) i els **basats en el gradient de la política** (com el PPO, que optimitza directament la política). Es reparteixen també entre les variants *on-policy* i *off-policy* (definides al capítol 2) i entre les dues biblioteques emprades (taula 5.2). Tots comparteixen la mateixa observació de 240 dimensions, la mateixa arquitectura MLP de [256, 256] sense extractor convolucional, i la mateixa funció d'avaluació; així qualsevol diferència s'atribueix a l'algorisme i no a l'arquitectura.

Algorisme	Biblioteca	Política	Paral·lisme
DQN-RLCard	RLCard	Off-policy	1 entorn seqüencial
NFSP-RLCard	RLCard	Off-policy + SL	1 entorn seqüencial
DQN-SB3	SB3	Off-policy	1 entorn
PPO-SB3	SB3	On-policy	48 entorns (SubprocVecEnv)

Taula 5.2: Els quatre algorismes comparats a la Fase 1.

Avaluació en dues òptiques. Com que els algorismes tenen velocitats molt diferents, comparar-los només per passos seria injust amb els paral·lelitzables. Per això es mesura el rendiment sota dues òptiques complementàries:

- **Per passos** (5 milions): la qualitat assolida després del mateix nombre de passos d'entrenament (eficiència per mostra).
- **Per temps real** (4 hores per agent): la qualitat assolida amb el mateix pressupost temporal (rendiment efectiu segons el *throughput*).

Oponents. Cada agent s'entrena contra una barreja d'oponents (taula 5.3) amb un criteri comú: una petita fracció de RandomAgent (per mantenir diversitat i explorar estats poc habituals) i un gruix d'AgentReg1es (el senyal estratègic principal). La diferència és el *self-play*: els agents de RLCard (DQN i NFSP) el suporten de manera

nativa, de manera que un 30% de les partides es juguen contra una versió del propi agent. En el cas del DQN, aquesta versió és una còpia *Polyak* que s'actualitza lentament (un 5% cap als pesos actuals cada 5 000 passos): així l'oponent millora amb l'agent però no canvia tan de pressa com per desestabilitzar l'aprenentatge.

Agent	Random	AgentRegles	Self-play
DQN-RLCard	10%	60%	30% (Polyak)
NFSP-RLCard	10%	60%	30% (natiu)
DQN-SB3	5%	95%	—
PPO-SB3	5%	95%	—

Taula 5.3: Distribució d'oponents per episodi a la Fase 1. Els agents de RLCard incorporen self-play; els de SB3 entrenen només contra oponents fixos (RandomAgent i AgentRegles).

Bucle d'entrenament. Els dos agents de RLCard s'entrenen amb un bucle manual: controla el format de cada transició que s'injecta al *replay buffer*, alterna la posició de l'aprenent a cada partida i resol l'assignació de crèdit quan és l'oponent qui tanca la partida (la recompensa terminal s'assigna a la darrera jugada de l'aprenent). Els agents SB3 usen la rutina nativa de la biblioteca amb un *callback* que avalua i desa el millor model cada 500 000 passos. PPO genera l'experiència amb 48 entorns en paral·lel repartits entre oponents i posicions. Per garantir que l'avaluació mesura generalització i no memorització, l'AgentRegles de l'avaluació s'instancia amb una llavor diferent de la d'entrenament.

Resultats. Cap agent assoleix un rendiment alt i estable alhora (taula 5.4). DQN-SB3 és el millor en totes òptiques (pic del 62,8% per passos i 58,5% per temps), però pateix una degradació severa cap al final de l'entrenament (de 62,8% a 36,5%): la seva millor mètrica és molt superior a la final. PPO-SB3 és el més feble a igualtat de passos (29,5%), perquè un algorisme on-policy necessita molts més passos per estabilitzar-se, però té un *throughput* incomparable (taula 5.5): genera 36,5 milions de passos per hora, 28 vegades més que NFSP. Aquest contrast fa que el rànquing canviï entre les dues òptiques. El patró comú a tots els agents és la **inestabilitat**: regredeixen cap al final de l'entrenament, senyal que el problema (partida llarga, recompensa molt diferida) no proporciona un senyal d'aprenentatge prou estable.

Agent	Per passos (5M)		Per temps (4 h)	
	Millor	Final	Millor	Final
DQN-SB3	62,8%	36,5%	58,5%	17,2%
NFSP-RLCard	53,2%	29,0%	43,0%	33,2%
DQN-RLCard	51,0%	25,8%	44,5%	34,2%
PPO-SB3	29,5%	18,5%	42,0%	21,2%

Taula 5.4: metric de la Fase 1 sota les dues òptiques. «Millor» és el pic durant l'entrenament; «Final» és el valor en acabar, que evidencia la inestabilitat.

Algorisme	Passos/hora	Factor vs NFSP
PPO-SB3	36,5 M	28×
DQN-SB3	7,0 M	5,4×
DQN-RLCard	5,5 M	4,2×
NFSP-RLCard	1,3 M	1×

Taula 5.5: *Throughput* dels quatre algorismes. El paral·lelisme de PPO-SB3 el fa 28 vegades més ràpid que NFSP-RLCard.

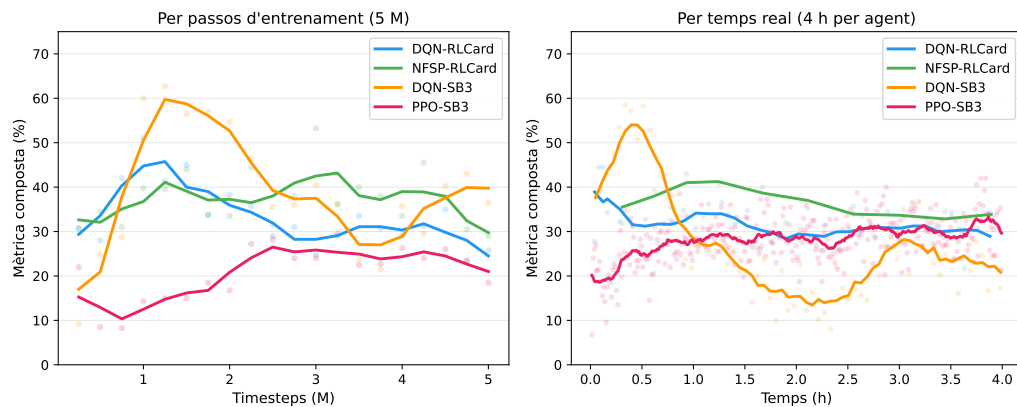


Figura 5.1: Mètrica composta de la Fase 1 sota les dues òptiques d'avaluació. A l'esquerra, rendiment als 5 M passos d'entrenament (eficiència per mostra); a la dreta, rendiment al llarg de 4 hores de còmput per agent (rendiment efectiu). DQN-SB3 lidera en ambdues, però el rànquing de la resta canvia: PPO-SB3 és el pitjor per passos però supera DQN-RLCard i NFSP-RLCard per temps gràcies al seu *throughput*.

Decisió. El coll d'ampolla no és el còmput sinó el **senyal d'aprenentatge**. El cas més clar és PPO-SB3: és l'agent més escalable i ràpid, però el que menys aprofita una partida llarga amb recompensa diferida. La hipòtesi que en sorgeix és que factoritzar la partida en unitats més curtes i denses (mans) hauria de donar un senyal molt més ric. Això motiva la Fase 2.

Nota sobre l'observació. Les Fases 1 i 2 van utilitzar una versió primerenca del vector de context de 23 dimensions (239 dimensions en total). A partir de la Fase 3 es va ampliar a 24 dimensions (240 en total), tal com es descriu al capítol 4. El canvi no afecta les conclusions comparatives, atès que totes les sèries d'una mateixa fase comparteixen la mateixa observació.

5.2.2 Fase 2: *curriculum learning* (mà → partida)

Hipòtesi. Entrenar directament sobre partides senceres obliga l'agent a resoldre alhora dos problemes acoblats: jugar bé cada mà (tàctica local) i gestionar el marcador al llarg de la partida (estratègia global). El *curriculum learning* [14], que presenta primer els exemples simples i després els complexos, permet desacoblar-los. Al Truc, una mà és la unitat natural simple: episodi curt (~15 passos) i recompensa calculable al final.

La hipòtesi és que entrenar primer sobre mans (amb l'entorn TrucGameMa/TrucEnvMa descrit al capítol 4) i després fer *finetune* sobre partides senceres accelera i millora l'aprenentatge respecte a entrenar-hi directament.

Disseny. *El curriculum de dues etapes.* S'ha dissenyat un curriculum que va de la tasca densa i curta a l'escassa i llarga (figura 5.2):

1. **Etapa de mans** (12M passos): l'agent s'entrena a TrucGameMa, on cada mà és un episodi independent (~15 passos) amb recompensa densa normalitzada (punts/24). Aquí aprèn la tàctica local: quan jugar fort, quan acceptar un envit, quan plantar el truc.
2. **Etapa de partides** (12M passos): els pesos apresos es transfereixen a TrucGame, on l'episodi és una partida sencera fins a 24 punts. L'agent parteix d'una política ja formada i el *finetune* recau sobre l'estratègia a llarg termini, com la gestió del marcador.

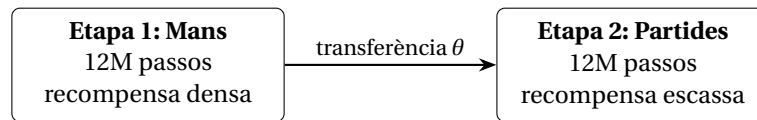


Figura 5.2: Currículum de dues etapes: mans individuals amb recompensa densa, seguides de partides senceres amb transferència dels pesos apresos (θ).

Comparació experimental. Per a cada algorisme s'executen dues configuracions amb el mateix pressupost total de 24 milions de passos: **control** (24M directament sobre partides) i **curriculum** (les dues etapes anteriors). Per aïllar l'efecte del curriculum s'elimina el *self-play* i tots els agents s'entrenen amb la mateixa distribució d'oponents (10% Random + 90% Regles). L'avaluació es fa sempre sobre partides senceres, encara que l'agent estigui aprenent sobre mans, perquè és on es vol mesurar la competència real.

Transferència al finetune. El pas de l'etapa de mans a la de partides no és automàtic: els pesos de la primera etapa (`best_mans`) es carreguen explícitament abans de la segona, via `PP0.load()` o carregant l'estat de la xarxa Q segons l'agent. A més, per al DQN-SB3 es redueix l'exploració inicial del *finetune* ($\epsilon = 0,10$ en lloc d'1,0) i s'escurça el *warmup*, perquè la política ja està formada i no cal tornar a explorar des de zero.

Riscos. El principal és l'**oblit catastròfic**: els algorismes on-policy com PPO descarquen les dades antigues a cada actualització, de manera que l'agent podria oblidar el que va aprendre sobre mans en exposar-se a les partides. Un segon risc és el desajust de distribució, ja que la distribució de mans amb el marcador a 0-0 difereix de la que apareix amb el marcador avançat.

Resultats. La taula 5.6 mostra que, a igualtat de pressupost (12M passos), entrenar sobre mans és *uniformement* superior a entrenar sobre partides per als quatre agents, amb diferències espectaculars per als agents SB3 (+42 i +56 punts percentuals). Aquesta és la validació clau: l'entorn per mans aporta un senyal d'aprenentatge qualitativament

5. METODOLOGIA D'ENTRENAMENT

millor. Ara bé, la taula 5.7 mostra que la transferència d'aquest aprenentatge a partides senceres (després del *finetune*) no és universal: només 2 de 4 agents en surten beneficiats al pressupost total. El coll d'ampolla passa a ser la transferència, no l'aprenentatge inicial.

Agent	Control 12M	Curriculum 12M (mans)	Δ
DQN-RLCard	18,2%	26,0%	+7,8
NFSP-RLCard	25,2%	27,2%	+2,0
DQN-SB3	44,0%	86,2%	+42,2
PPO-SB3	30,8%	87,2%	+56,5

Taula 5.6: Comparació a 12M passos (mateix pressupost, avaluació sobre partides). El valor *metric* entrenant sobre mans és uniformement superior.

Agent	Control 24M	Curriculum (12M+12M)	Δ
DQN-RLCard	52,0%	22,2%	-29,8
NFSP-RLCard	55,5%	60,0%	+4,5
DQN-SB3	60,5%	56,0%	-4,5
PPO-SB3	35,0%	75,0%	+40,0

Taula 5.7: Comparació a 24M passos totals (control sencer vs curriculum complet). Només PPO-SB3 i NFSP-RLCard milloren; DQN-RLCard col·lapsa per oblit catastròfic.

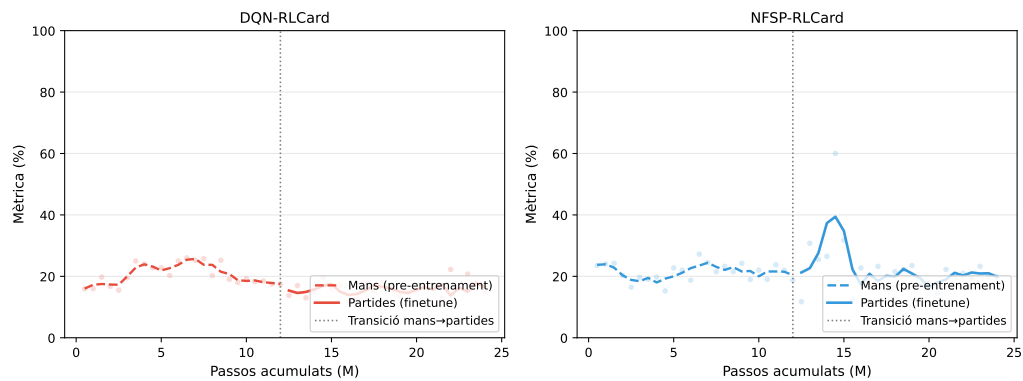


Figura 5.3: Corbes del *curriculum* per als agents RLCARD. La línia ratllada és la fase de mans (pre-entrenament); la línia sòlida el *finetune* sobre partides. La línia puntejada marca la transició als 12M passos. DQN-RLCard col·lapsa al *finetune*; NFSP-RLCard recupera i millora lleugerament.

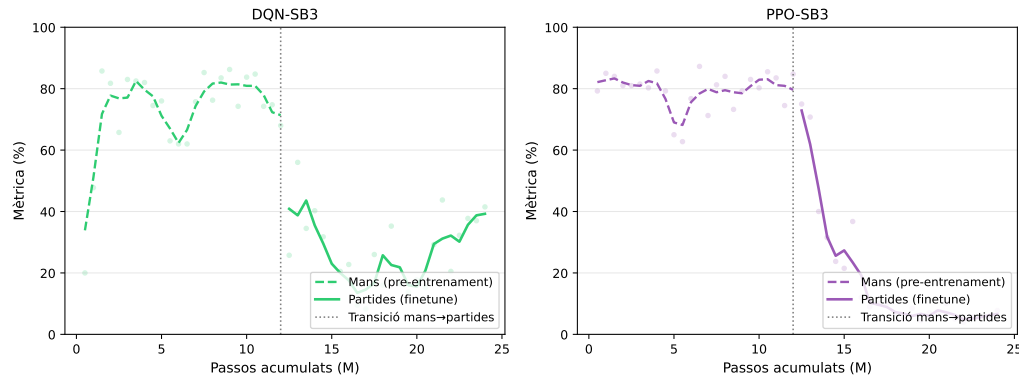


Figura 5.4: Corbes del *curriculum* per als agents SB3. DQN-SB3 assoleix un pic alt en mans (86%) però perd rendiment al *finetune*. PPO-SB3 manté el nivell i tanca a 75%, el millor resultat del treball fins al moment.

El cas més destacat és **PPO-SB3**, l'agent més feble de la Fase 1: passa del 35% al 75% amb el *curriculum* complet (+40 punts), el millor resultat del treball fins aleshores. Es valida que el problema de PPO no era l'algorisme sinó l'horitzó llarg amb recompensa diferida. En canvi, **DQN-RLCard** col·lapsa al *finetune* (22,2%): la seva implementació no integra bé la càrrega de pesos previs i, sense el *self-play* regularitzador de la Fase 1, perd estabilitat.

Decisió. El *curriculum learning* millora el rendiment, però de manera condicionada a l'agent: a igualtat de 12M passos tots quatre agents aprenen molt millor sobre mans, però després del *finetune* només PPO-SB3 (massivament) i NFSP-RLCard (lleugerament) en surten beneficiats al pressupost total. El coll d'ampolla, doncs, és la **transferència** de mans a partides, no l'aprenentatge inicial; i el *curriculum* queda validat com a tècnica clau per a PPO-SB3.

5.2.3 Checkpoint 1: models seleccionats

Tancades les Fases 1 i 2, cal decidir quins dels quatre models continuen a la resta del treball. El criteri combina el rendiment (*metric*) i el cost d'entrenament, i el resultat és clar: avancen els dos agents SB3 i es descarten els dos de RLCard.

PPO-SB3 és el gran beneficiari del *curriculum* i passa a ser el *baseline* de referència. Del 35% puja al 75% amb el *curriculum* complet (el millor resultat del treball fins aleshores) i és el millor aprenent sobre mans (87,2%), amb aproximadament la meitat del cost de DQN-SB3 (~40 min davant de ~5,5 h per execució). La seva narrativa resumeix la lliçó del bloc: PPO no tenia un problema de capacitat sinó de senyal d'aprenentatge, i el *curriculum* el resol. El seu *throughput* excel·lent permet, a més, iterar ràpidament a les fases següents. **DQN-SB3** es manté com a *baseline* robust: és el millor sobre partides sense *curriculum* (60,5%), té un sostre del 86,2% sobre mans i representa la família dels algorismes basats en la funció de valor, el contrapunt natural a PPO; el seu cost (~5,5 h per execució) és assumible per als experiments llargs de les fases següents.

Es descarten, en canvi, els dos agents de RLCard. **DQN-RLCard** pateix un oblit catastròfic sever al *finetune* (col·lapsa al 22%) i és consistentment inferior a DQN-SB3

en tots els experiments. **NFSP-RLCard** és l'algorisme més lent (~17 h per execució), no supera el 60% de *metric* i no mostra cap escalabilitat amb més còmput: el benefici és massa modest per al cost.

5.3 Bloc 2: representació de l'estat i memòria

Amb els dos agents SB3 seleccionats al Checkpoint 1, el segon bloc explora dues direccions ortogonals per superar el sostre de rendiment: millorar la *representació* de l'estat (Fase 3, dividida en arquitectura i inicialització del cos) i dotar l'agent de *memòria* entre mans (Fase 4). El Checkpoint 2 tanca el bloc fixant el model que s'adopta per a desplegament.

5.3.1 Fase 3: el feature extractor COS

La Fase 3 estudia un *feature extractor* convolucional propi, anomenat COS (CosMultiInput), com a alternativa a l'extractor pla per defecte de SB3. L'estudi es divideix en dues preguntes acoblades, tractades en dues subsubseccions: si l'**arquitectura** convolucional aporta valor per si sola (Q1, secció 5.3.1.1) i, en cas que la representació importi, si **inicialitzar-la** amb coneixement previ del joc desbloqueja un rendiment superior (Q2, secció 5.3.1.2).

5.3.1.1 Arquitectura: CNN vs MLP

Hipòtesi. El vector pla de 240 dimensions amaga l'estructura espacial del tensor de cartes. Un MLP pla ha de *redescobrir* per ell mateix que el 4 de Copes i el 4 d'Ors comparteixen rang, o que cartes de pals diferents poden tenir forces relacionades. La hipòtesi (Q1) és que un extractor que respecta l'estructura $6 \times 4 \times 9$ del tensor de cartes (mitjançant convolucions) supera el MLP estàndard.

Disseny. *L'extractor CosMultiInput.* Per provar la hipòtesi es dissenya CosMultiInput (RL/models/core/feature_extractor.py), un extractor de dues branques (figura 5.5). La branca Convolutional Neural Network (CNN) aplica dues convolucions sobre el tensor (6,4,9) (taula 4.1): la primera amb nucli 1×3 detecta patrons entre rangs consecutius; la segona amb nucli 3×3 integra la informació entre pals i rangs, produint 320 dimensions. La branca densa projecta el vector de context de 24 dimensions (taula 4.2) a 32. Les dues sortides es concatenen i es projecten a 256 dimensions. La sortida coincideix amb l'amplada de la primera capa oculta de la política (net_arch=[256,256]), de manera que CosMultiInput substitueix l'extractor per defecte de SB3 (CombinedExtractor, que simplement aplanava l'observació) sense cap altre canvi. L'adaptador CosMultiInputSB3 (RL/models/sb3/sb3_features_extractor.py) fa de pont entre el vector pla i les dues branques.

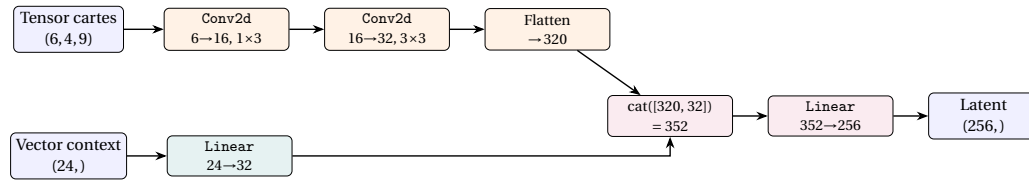


Figura 5.5: CosMultiInput: arquitectura de dues branques. La branca CNN processa el tensor de cartes (6, 4, 9); la branca lineal el vector de context (24,). Les sortides es concatenen i es projecten a un vector latent de 256 dimensions.

Els tres protocols. La comparació es fa sobre els tres protocols d'entrenament introduïts a la Fase 2 (secció 5.2.2), que es recorden aquí perquè ara es comparen tots tres:

- **Control:** entrenar directament sobre *partides senceres*; la condició de referència, sense cap facilitació.
- **Curriculum:** entrenar primer sobre *mans* i després fer *finetune* sobre partides senceres (el currículum de dues etapes de la Fase 2).
- **Mans:** entrenar *exclusivament* sobre mans individuals, sense el pas posterior a partides.

Comparació experimental. Per a cada protocol i agent es compara la variant COS (amb pesos aleatoris) amb la variant MLP, amb 24 milions de passos cada combinació i la mateixa distribució d'oponents que la Fase 2 (10% Random + 90% Regles). Això suposa sis runs nous amb COS i dos amb MLP sobre mans; les quatre sèries MLP de control i curriculum es reutilitzen de la Fase 2 per no reentrenar-les.

Resultats. El COS amb pesos aleatoris no supera sistemàticament el MLP (taula 5.8). Només aporta una millora clara al protocol de control per a DQN-SB3 (+10,5 punts). El protocol **mans** torna a ser el millor en termes absoluts (totes les combinacions superen el 82%), però la diferència entre COS i MLP hi és marginal (< 4 punts). La inducció estructural per si sola, amb pesos aleatoris, no és suficient per desbloquejar un avantatge. La lectura és coherent: al protocol de control, sense cap facilitació, la capacitat addicional de la CNN per extreure patrons de l'estructura de les cartes es tradueix en un guany real; al protocol de mans, el currículum d'entrenament sobre mans individuals ja exposa l'agent als patrons carta-a-carta en un context simplificat, de manera que el MLP arriba al protocol de mans amb una representació prou rica i la CNN no hi aporta res addicional.

Protocol	Agent	COS	MLP	Δ
Control	DQN-SB3	71,0%	60,5%	+10,5
Control	PPO-SB3	32,5%	35,0%	-2,5
Curriculum	DQN-SB3	56,2%	56,0%	+0,2
Curriculum	PPO-SB3	63,0%	75,0%	-12,0
Mans	DQN-SB3	82,2%	85,8%	-3,6
Mans	PPO-SB3	89,0%	87,2%	+1,8

Taula 5.8: Pic de metric (24M passos) per a COS amb pesos aleatoris vs MLP.

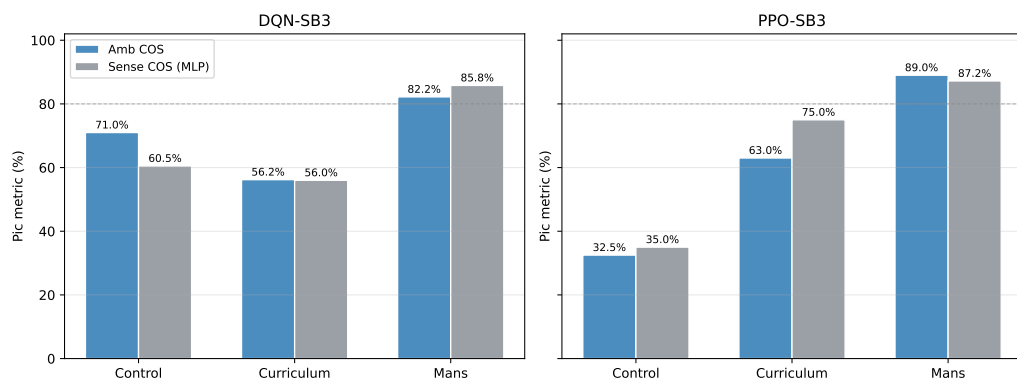


Figura 5.6: Pic de metric (24M passos) per protocol i agent, COS amb pesos aleatoris vs MLP. La línia discontinua marca el 80%. El COS només supera clarament el MLP al control de DQN-SB3; al protocol de mans (el millor en absolut) la diferència és marginal.

Decisió. Es fixa el protocol de mans per a la resta del bloc. Com que el COS amb pesos aleatoris no arrenca amb cap avantatge representacional, la pregunta natural és si *inicialitzar-lo* amb coneixement previ del joc canvia el resultat. Això motiva la subsubsecció següent.

5.3.1.2 Inicialització supervisada del cos

Hipòtesi. Un cos preentrenat que ja distingeix rang, pal i força de les cartes des del primer pas d'AR permet a la política concentrar-se en la presa de decisions en lloc de reaprendre la representació. La pregunta (Q2) és doble: aporta valor el preentrenament supervisat, i quin règim (congelat o ajustat) és preferible?

Disseny. *El preentrenament supervisat.* La idea és donar al cos `CosMultiInput` una representació útil de les cartes *abans* que comenci l'AR, per tal que l'agent no hagi de descobrir l'estructura del joc i la política al mateix temps. Per aconseguir-ho, s'afegeix temporalment al cos el model `ModelPreEntrenament`, que hi acobla tres caps de predicció (taula 5.9): la força de cada carta de la mà, la màscara d'accions legals i la puntuació d'envit (figura 5.7).

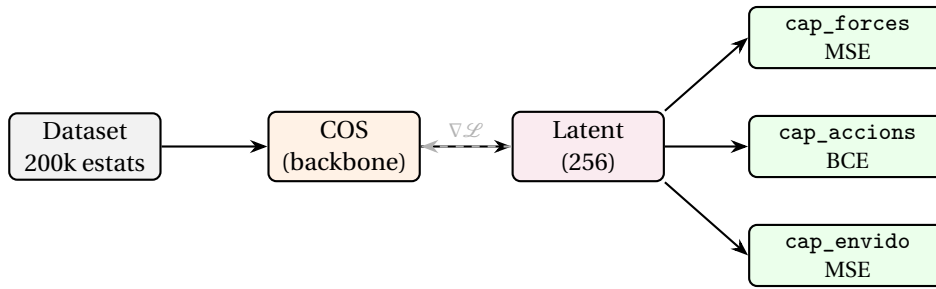


Figura 5.7: Esquema del preentrenament supervisat: els tres caps propaguen gradient al cos i l'obliguen a aprendre representacions sensibles a l'estructura del joc. Un cop acabat, els caps es descarten i es conserven només els pesos del cos.

El dataset es genera jugant partides amb una política pseudo-aleatòria sobre l'entorn real: a cada pas es registra l'observació de l'agent i s'extreuen automàticament les etiquetes del propi motor de joc (la força de cada carta via TrucJudger, la màscara d'accions legals de l'estat actual i els punts d'envit de la mà inicial). El resultat són 200 000 transicions amb un repartiment 80/20 entre entrenament i validació; la pèrdua combinada pondera els tres objectius ($1,0 \cdot \mathcal{L}_{\text{envit}} + 2,0 \cdot \mathcal{L}_{\text{accions}} + 2,5 \cdot \mathcal{L}_{\text{forces}}$), donant més pes als objectius que exigeixen una comprensió més fina de l'estructura de les cartes. Un cop acabat l'entrenament supervisat, els caps es descarten; la seva única funció era proporcionar gradient informatiu al cos, i el que es conserva és el coneixement que el cos ha consolidat als seus pesos.

Cap	Sortida	Mida	Pèrdua
cap_envido	Punts d'envit de la mà	1	MSE
cap_accions_legals	Màscara d'accions legals	19	BCE
cap_forces	Força de cada carta a la mà	3	MSE

Taula 5.9: Caps de predicció del ModelPreEntrenament per al preentrenament supervisat del cos CosMultiInput.

Els quatre règims. El coneixement preentrenat es pot aprofitar de maneres diferents, que constitueixen la variable d'estudi de la fase:

- **Sense COS (MLP):** l'extractor pla per defecte de SB3, sense cap inducció estructural; estableix el sostre d'un model sense coneixement de l'estructura de les cartes.
- **Scratch** (des de zero): el COS amb *pesos aleatoris*; la CNN ha d'aprendre la representació de les cartes alhora que la política.
- **Frozen** (congelat): el COS amb *pesos preentrenats supervisadament i congelats*; la representació és fixa i estable, i només s'entrena la política.
- **Finetune** (ajust fi): el COS amb pesos preentrenats que *es continuen ajustant* durant l'AR; té millor potencial però arrisca a oblidar la representació inicial.

Cada comparació aïlla un efecte: *scratch* contra MLP mesura l'efecte de l'arquitectura, i *frozen/finetune* contra *scratch* mesuren l'efecte del preentrenament.

Resultats. El règim **frozen és el millor** (taula 5.10). Per a DQN-SB3 assoleix el 91,2% (+5,4 punts sobre el MLP, +9 sobre *scratch*): el cos preentrenat i congelat actua com a extractor estable que permet a la xarxa de valor aprendre sense interferències. El *finetune* degrada respecte al *frozen* en tots dos agents: el gradient de l'AR deteriora parcialment la representació preentrenada. Per a PPO-SB3 els quatre règims convergeixen entre 87–89%, de manera que el preentrenament hi aporta poc.

Règim	DQN-SB3	PPO-SB3
Sense COS (MLP)	85,8%	87,2%
Scratch	82,2%	89,0%
Frozen	91,2%	89,5%
Finetune	88,5%	88,0%

Taula 5.10: Pic de *metric* (protocol mans, 24M passos) per als quatre règims d'inicialització del cos.

Decisió. El preentrenament supervisat aporta valor real, però només quan el cos queda **congelat**. S'adopta el COS preentrenat i congelat com a *backbone* estable per a la fase següent.

5.3.2 Fase 4: memòria d'oponent (LSTM) i pool d'oponents

Hipòtesi (Q3). Totes les fases anteriors entrenen contra un oponent fix i amb un agent sense estat intern. La Fase 4 introdueix dues variables: una memòria seqüencial (Long Short-Term Memory (LSTM)) que manté estat entre mans, i un *pool* de sis variants d'oponent (taula 5.1). La hipòtesi és que l'LSTM, entrenada en sessions de $N = 5$ mans consecutives contra el mateix oponent (entorn `TrucGymEnvSessio`), aprendrà a detectar el patró de l'oponent i s'hi adaptarà, amb un win-rate creixent dins la sessió.

Disseny. *El pool d'oponents.* En lloc d'un oponent fix, l'agent s'entrena contra sis variants d'`AgentRegles` amb estils contrastats (taula 5.1). La diversitat és deliberada: amb un únic oponent existiria una política mitjana òptima que un agent sense memòria podria memoritzar; amb sis estils clarament diferents no n'hi ha cap, de manera que l'única manera de jugar bé contra tots és *detectar* qui es té al davant i adaptar-s'hi.

Sessions multi-mà. Per donar marge a aquesta adaptació, l'episodi d'AR deixa de ser una mà i passa a ser una sessió de $N = 5$ mans consecutives contra el mateix oponent. El nou entorn `TrucGymEnvSessio` envolta `TrucGymEnvMa` i *intercepta* els senyals de final (`terminated`) intermedis, de manera que l'estat de l'LSTM no es reinicia fins al final de les cinc mans; al començament de cada sessió es mostreja un oponent nou del pool.

L'arquitectura amb memòria. La cadena de processament és $\text{obs}(240) \rightarrow \text{COS congelat}(256) \rightarrow \text{LSTM}(256) \rightarrow \text{MLP lleuger} \rightarrow \text{caps de política i valor}$. S'usa `RecurrentPPO` (de `sb3_contrib`) amb *retropropagació en el temps truncada* sobre seqüències, on el

`batch_size` es mesura en seqüències i no en transicions individuals. No s'usa DRQN (l'anàleg recurrent del DQN) perquè requeriria un *replay buffer* seqüencial no disponible a SB3.

Pressupost i avaluació. Es comparen dos runs nous amb les línies base de la Fase 3.5: **F4-ablació** (PPO + COS congelat + pool, sense memòria) i **F4-complet** (RecurrentPPO amb LSTM sobre el COS congelat + pool); la primera comparació aïlla l'efecte del pool i la segona, l'efecte de la memòria. Cada run nou s'entrena 12M passos, la meitat que a la Fase 3.5, perquè la retropropagació en el temps encareix molt el còmput. L'avaluació, a més de la *metric* habitual, acumula el win-rate *per posició dins la sessió* (de la mà 1 a la 5) per detectar si l'agent millora a mesura que coneix l'oponent.

Resultats. Cap de les tres hipòtesis es valida plenament (taula 5.11):

- **H1 falla:** l'LSTM no millora el rendiment global. F4-complet (82,0%) queda 4 punts *per sota* de F4-ablació (86,0%), amb un cost computacional 46× superior (15,2 h vs 0,33 h).
- **H2 falla:** no hi ha adaptació clara dins la sessió. La corba de win-rate per posició és sorollosa i no monòtona ([77,5; 73,2; 80,7; 71,1; 78,9], $\Delta = +1,4$ punts) i el mateix patró apareix a la versió sense memòria, cosa que indica un artefacte de la mida de mostra (~56 sessions) i no aprenentatge adaptatiu real.
- **H3 parcial:** F4-ablació queda 3,5 punts sota PPO frozen, just al límit del criteri (± 3 punts); el pool dificulta lleugerament la convergència sense memòria.

La causa principal del fracàs de l'LSTM és el **desajust entre entrenament i avaluació**: s'entrena sobre sessions de 5 mans (~50 passos), però s'avalua sobre sessions de 5 partides senceres (~500 passos), un context molt més llarg que la xarxa no ha vist mai durant l'entrenament. S'hi sumen el pressupost reduït (12M vs 24M de les línies base) i les sessions d'entrenament massa curtes perquè la retropropagació propagui senyals d'adaptació.

Run	Memòria	Pic <i>metric</i> (12M)	Temps
DQN frozen (F3.5)	No	87,75%	—
PPO frozen (F3.5)	No	89,5%	0,33 h
F4-ablació	No	86,0%	0,33 h
F4-complet	LSTM 256	82,0%	15,2 h

Taula 5.11: Resultats de la Fase 4 (pressupost igualat a 12M passos). DQN frozen mostra el seu màxim dins el pressupost de 12M; el seu pic global és del 91,2% a 23M.

Decisió. L'LSTM no justifica el seu cost amb el disseny actual, de manera que la memòria queda descartada per al model final. La tria entre els candidats restants (F4-ablació i les línies base de la Fase 3.5) es resol al Checkpoint 2.

5.3.3 Checkpoint 2: model seleccionat

Tancades les Fases 3 i 4, el model que s'adopta per a desplegament és **F4-ablació** (PPO + COS congelat + pool, 86,0% a 12M passos). La decisió pondera rendiment, robustesa i cost.

La raó principal és la **robustesa davant estils diversos**. F4-ablació s'ha entrenat contra les sis variants d'AgentRegles, de manera que la seva política és generalista; les millors línies base de la Fase 3.5 (DQN i PPO frozen) van entrenar sempre contra un oponent fix i no s'han enfrontat a aquesta diversitat. En un context de desplegament real, on l'oponent és desconegut, aquesta generalitat és més valuosa que uns quants punts de win-rate aconseguits contra un únic adversari. A això s'hi suma la **rapidesa d'entrenament** (0,33 h per execució, davant les 15,2 h de F4-complet o les diverses hores del DQN), que permet iterar i reentrenar amb facilitat, i un **rendiment competitiu** al pressupost igualat (només 3,5 punts sota PPO frozen).

DQN frozen té el millor win-rate brut (91,2% al pic global), però va ser entrenat contra un oponent fix i no s'ha provat contra el pool; per això no es tria. L'extensió natural seria entrenar DQN amb el pool d'oponents per combinar el seu rendiment amb la robustesa, però no s'ha executat per limitacions de temps. Finalment, l'LSTM (F4-complet) queda descartada: un cost 46× superior i un rendiment 4 punts inferior no compensen, i cap de les seves hipòtesis s'ha validat.

Aquesta robustesa, però, encara no s'ha *quantificat*: la *metric* clàssica promitja entre oponents i pot amagar debilitats davant estils concrets. Mesurar-la i millorar-la és l'objectiu del Bloc 3.

CAPÍTOL 6

CONCLUSIONS

[Conclusions del treball: resum del context i del problema, principals conclusions, implicacions teòriques i pràctiques, limitacions i treball futur.]



HIPERPARÀMETRES DELS MODELS

[Taules amb els hiperparàmetres complets de cada algorisme (DQN-RLCard, NFSP-RLCard, DQN-SB3, PPO-SB3) per a cada fase experimental.]

APÈNDIX

RESULTATS COMPLETS PER FASE

[Taules amb tots els valors numèrics dels experiments: mètriques finals, temps d'entrenament, etc.]

BIBLIOGRAFIA

- [1] J. Huizinga, *Homo Ludens: A Study of the Play-Element in Culture*. Routledge & Kegan Paul, 1938. 1
- [2] M. Campbell, A. J. Hoane Jr., and F.-h. Hsu, “Deep Blue,” *Artificial Intelligence*, vol. 134, no. 1–2, pp. 57–83, 2002. 1.1
- [3] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, pp. 484–489, 2016. 1.1
- [4] C. Berner *et al.*, “Dota 2 with large scale deep reinforcement learning,” 2019, arXiv:1912.06680. 1.1
- [5] N. Brown and T. Sandholm, “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals,” *Science*, vol. 359, no. 6374, pp. 418–424, 2018. 1.2, 2.2
- [6] —, “Superhuman AI for multiplayer poker,” *Science*, vol. 365, no. 6456, pp. 885–890, 2019. 1.2, 2.2, 2.3.4
- [7] Retruc.net, “Reglament del truc,” <https://retruc.net/reglamenttruc>, 2024, accedit: 2026. 1.3
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. 2.1
- [9] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015. 2.3.1
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. 2.3.2
- [11] M. Hausknecht and P. Stone, “Deep recurrent Q-learning for partially observable MDPs,” *arXiv preprint arXiv:1507.06527*, 2015. 2.3.3
- [12] G. W. Brown, “Iterative solution of games by fictitious play,” in *Activity Analysis of Production and Allocation*, T. C. Koopmans, Ed. Wiley, 1951, pp. 374–376. 2.3.4
- [13] J. Heinrich and D. Silver, “Deep reinforcement learning from self-play in imperfect-information games,” *arXiv preprint arXiv:1603.01121*, 2016. 2.3.4
- [14] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, 2009, pp. 41–48. 5.2.2